

Dokumentation zur
betrieblichen Projektarbeit:



***Konzeption und Entwicklung einer
Schnittstelle zum Datenimport von über
Drittanbieter-Systeme erfassten
Mitgliederbestandsmeldungen in das
IntelliVerband-Portal***

IHK Abschlussprüfung

Winter 2015/2016

Ausbildungsberuf:

Fachinformatiker - Anwendungsentwicklung

Ausbildender Betrieb:

XYZ GmbH & Co. KG
Musterstraße
12345 Musterstadt

Projektverantwortlicher:
Herr Vorname Nachname
vnname@firma.de

Prüfungsteilnehmer:

Herr M
Zu Hause 1
12345 Musterstadt

Azubi-ID 0000000000
Geb. Datum: 30.02.1970

Inhalt

1. AUSGANGSSITUATION	5
1.1 PROJEKTUMFELD	5
1.1.1 Firma	5
1.1.2 Entwicklungsabteilung	5
1.1.3 Kunden	5
1.1.4 Softwaresystem	5
1.1.5 Schnittstellen	5
1.2 KUNDENWÜNSCHE	6
1.2.1 Anforderung	6
1.2.2 Auftraggeber	6
1.2.3 Referenzkunden	6
1.3 PROJEKTZIELE	6
1.4 IST-ZUSTAND	6
1.5 SOLL-ZUSTAND:	6
1.6 TEILAUFGABEN	7
1.6.1 Erstellung Importformatvorlage	7
1.6.2 Programmierung Import	7
1.6.3 Datenaggregation	8
2. RESSOURCEN UND ABLAUFPLAN	8
2.1 PERSONALPLANUNG	8
2.2 SACHMITTELPLANUNG	8
2.3 KOSTENPLANUNG	8
2.4 TERMINPLANUNG	9
2.5 ABLAUFPLAN	9
3. DURCHFÜHRUNG.....	10
3.1 ANALYSE UND KONZEPTION	10
3.1.1 SOLL-/IST-Analyse: Vorgaben des DOSB	10
3.1.2 IST-/SOLL-Analyse: IntelliVerband-System	10
3.1.3 Konzeption Importdateiformat	10
3.1.4 Abnahme Dateivorlage durch internen Auftraggeber.....	11
3.1.5 Validierung durch externen Kunden	11
3.1.6 Re-Analyse und Änderung Dateiformat	11
3.1.7 Detailanalyse Datenfluss und Tabellen	12
3.1.8 Konzeption und Planung Programmablauf	13
3.2 UMSETZUNG UND IMPLEMENTIERUNG	13
3.2.1 Erstellung der Datei-Vorlage	14
3.2.2 Programmierung des Datei Imports	14
3.2.3 Programmierung Datenverarbeitung Geschäftslogik	15
3.2.4 Programmierung der Aggregation nach Altersgruppen	19
3.2.5 Integrationstest	19
3.2.6 Deployment	19
3.3 QUALITÄTSSICHERUNG	19
3.3.1 Begleitende Tests	19

3.3.2 Test mit Pilotkunden	20
3.3.3 Ergebniskontrolle	20
3.3.4 Abnahme durch Auftraggeber	20
3.3.5 Nacharbeiten	21
3.4 DOKUMENTATION	21
3.4.1 Anwenderdokumentation	21
3.4.2 Kundendokumentation	21
3.4.3 Projektdokumentation	21
3.5 ABWEICHUNGEN UND ANPASSUNGEN	21
3.5.1 Anpassungen am Importdateiformat	21
3.5.2 Anpassungen der DataSets	22
4. PROJEKTERGEBNIS	23
4.1 SOLL-/IST-VERGLEICH	23
4.2 ABWEICHUNGEN	23
4.3 QUALITÄTSKONTROLLE	24
4.4 FAZIT	24
4.5 AUSBLICK / ZUKUNFT	24
5. QUELLENVERZEICHNIS	25
6. ANHANG	26
A. PROJEKTABLAUFPLAN	26
B. DATENMODELL BST-MODUL	27
C. ANWENDERDOKUMENTATION	28
D. PERSÖNLICHE ERKLÄRUNG	36

Tabellen- und Abbildungsverzeichnis

TAB. 1: SACHMITTELKOSTEN KALKULATORISCH.	9
TAB. 2: ABWEICHUNGEN PROJEKTDAUER SOLL - IST.	23
ABB. 1: BESTANDSMELDUNG, DATEITRANSFER UND VERARBEITUNG ALS GESAMTPROZESS.	7
ABB. 2 A-C: DATEIVORLAGE FÜR IMPORT MIT JAHRGÄNGEN IN 220 SPALTEN (AUSSCHNITT). ...	11
ABB. 3: DATEIVORLAGE FÜR DEN IMPORT DER JAHRGÄNGE IN ZEILEN.	12
ABB. 4: DATENFLUSS: IMPORTDATEI, FREMDSCHLÜSSELERMITTLUNG UND ZIELTABELLEN.	12
ABB. 5: MAIN-METHODE MIT EINFACHEM SELECT CASE FÜR AUFRUFPARAMETER.	15
ABB. 6: CODE ZUR PRÜFUNG DER WERTAUSPRÄGUNGEN IN SPALTE "NUMMER"	16
ABB. 7: METHODE ZUR PRÜFUNG DER FREMDSCHLÜSSELREFERENZEN.	17
ABB. 8: ÜBERFÜHRUNG DER DATATABLE IN SORTIERTE DATAVIEW.	18
ABB. 9: VERARBEITUNGSDAUER VON 2 JOBS VOR UND NACH CODE-OPTIMIERUNG.	20
ABB. 10: PROJEKTABLAUF (GANTT-DIAGRAMM)	26
ABB. 11: DATENMODELL BESTANDSDATEN (AUSSCHNITT)	27

Abkürzungsverzeichnis und Glossar

.NET	Dot Net Framework - Software-Plattform der Firma Microsoft für die Anwendungsentwicklung.
ACID	Akronym: Atomicity, Consistency, Isolation, Durability - Prinzip von Datenbankzugriffen in Transaktionen, die als eine (atomare) Einheit aufgefasst und isoliert durchgeführt werden um konsistente Datenzustände dauerhaft zu speichern.
AppLog	Tabelle in der IntelliVerband-Datenbank, die als Protokoll für Operationen auf der Datenbankapplikation dient.
A-Zahl	Anzahl von Mitgliedern eines Vereins insgesamt, die qua Vereinsmitgliedschaft auch Mitglied im Landessportbund sind.
B-Zahl	Anzahl von Mitgliedern eines Vereins, die aufgrund Ihrer praktizierten Sportart einem bestimmten Fachverband zugeordnet werden; bei Ausübung mehrerer Sportarten auch Zuordnung zu mehreren Fachverbänden.
BOL	Business Object Layer – Geschäftsprozess-Schicht. In dieser Schicht werden die Klassen mit den Methoden der Geschäftslogik implementiert.
CSV	Comma Separated Values - kommaseparierte Werte; verbreitetes einfaches Dateiformat zur Datenspeicherung.
DAL	Data Object Layer - Datenbankzugriffsschicht. Objekte dieser Schicht, werden von den BOL-Klassen angesprochen und kommunizieren ihrerseits mit der Datenbank.
DOSB	Deutscher Olympischer Sportbund - Dachverband des Sports in der Bundesrepublik Deutschland.
Image	Binäres Speicherabbild einer Datei.
IntelliOnline	Sammelbegriff für die web-basierten Software-Produkte der Xyz für Vereine und Verbände.
IntelliVerband	Produktbezeichnung der Verbandsverwaltungs-Software – mit Datenbank und Web-Benutzeroberfläche (IntelliVerband-Portal)
Xyz	Xyz GmbH & Co. KG – Hersteller von IntelliVerband; Ausbildungsbetrieb in dem das Projekt durchgeführt wurde.
LSBH	Landessportbund Hessen - Verband mit Sportartübergreifenden Aufgabenstellungen; das Gegenteil sind (Sport)Fachverbände.
SportBiz	Ehemaliger Produktname der IntelliVerband-Software; findet sich im Namespace älterer Programmkomponenten wieder.
SQL, T-SQL	Structured Query Language - Sprache zur Abfrage / Manipulation von Daten in Datenbanksystemen (T-SQL = Microsoft Dialekt).
Use Case	Anwendungsfall für die prototypische Interaktion eines Benutzers mit einem (datenverarbeitenden) System.
VB, VB.NET	Visual Basic - Programmiersprache von Microsoft (im .NETFramework).

XLS, XLSX	Dateiformate / Dateiendungen von Microsoft Excel; technisch unterschiedliche ältere (97-2003) und neuere (2007 – auf XMLBasis) Version.
XML	eXtensible Markup Language - Auszeichnungssprache zur Darstellung hierarchisch strukturierter Daten in Form von Textdateien.

1. AUSGANGSSITUATION

1.1 Projektumfeld

1.1.1 Firma

Die XYZ GmbH & Co. KG ist ein System- und Software-Haus aus Musterstadt mit über 25 Jahren Erfahrung als IT-Dienstleister und mit namhaften Referenzen. Das Projekt wird in den Räumen der xyz am PC-Arbeitsplatz des Prüfungsteilnehmers durchgeführt.

1.1.2 Entwicklungsabteilung

Im Bereich der Software-Entwicklung hat sich ein Schwerpunkt bei der Entwicklung eigener Systeme für die Verwaltung von Vereinen und Verbänden herauskristallisiert. Im Team WebSolutions werden die internet-basierten Software-Produkte der *IntelliOnline*-Familie auf Basis von Microsoft-Technologien (VB.NET, C#, SQL Server und SharePoint) entwickelt, kundenindividuell angepasst und erweitert.

1.1.3 Kunden

Die Kunden der *IntelliVerband*-Software sind zumeist Organisationen aus dem Non-ProfitBereich. Ein Großteil davon sind Sportbünde der Landesebene, Sportfachverbände im Land und Bund sowie auch der Deutsche Olympische Sportbund (DOSB) als Dachverband des Sports in Deutschland.

1.1.4 Softwaresystem

Die von Xyz entwickelte *IntelliVerband*-Software ist eine webbasierte und modular aufgebaute Portal-Lösung, die speziell auf die Belange des Sports zugeschnitten ist. Für die Verwaltung eines Sportverbandes und seiner Mitgliedsorganisationen stehen 10 Module mit spezifischer Funktionalität zur Verfügung. Den Kern macht eine Stammdatenverwaltung für Organisationen und Personen mit Adressen aus. Weitere Module beinhalten typische Verwaltungsbereiche des Sports wie Gremien, Veranstaltungen und Lizenzen. Das Modul 'Bestandsdaten' umfasst schließlich Statistiken zu den Mitgliedsvereinen, der Mitgliederstruktur (Alter, Geschlecht) und den ausgeübten Sportarten.

Die Projektarbeit ist im Modul „Bestandsdaten“ der *IntelliVerband*-Software angesiedelt und umfasst den Import und die Verarbeitung von Daten der jährlichen Bestandsmeldungen der Vereine an Ihre übergeordnete Organisation, i.d.R. den Landessportverbänden.

1.1.5 Schnittstellen

Eine externe Schnittstelle für den Datenimport in *IntelliVerband* zu schaffen, ist das erklärte Ziel des Projekts: Mitgliederbestandsdaten, die über Drittanbieter-Systeme erhoben wurden, sollen für die Nutzung in der eigenen Software eingelesen werden. Die Drittanbieter-Systeme sind hierbei – wie auch das *intelliverband*-eigene Bestandserhebungsverfahren – an die inhaltlichen und technischen Vorgaben des DOSB gebunden.

Systemintern bestehen Schnittstellen zu den Organisationen im Kernmodul, sowie innerhalb des Bestandsmoduls zu Tabellen mit aggregierten Zahlen für Berichte nach Altersgruppen.

Schnittstellen zu anderen Personen ergeben sich aus meinen Aufgaben im Projekt. Diese sind: Die Analyse der Bestandserhebungsdaten und Anforderungen der Kunden, Spezifikation des Importformats und die Konzeption der Verarbeitungslogik (mit Abnahme durch den Projektverantwortlichen). Der Test mit einem Pilotkunden (extern) sowie die Anwenderdokumentation und Kurzeinweisung für Kollegen (interne Schnittstellen) schließen das Projekt ab.

1.2 Kundenwünsche

1.2.1 Anforderung

Die zu realisierende Importschnittstelle stellt eine generische Anforderung an das System dar, die von Kunden allgemein gewünscht ist. Die Umsetzung erfolgt losgelöst von einem direkten Kundenauftrag und dient auch der Festigung der Marktposition der *IntelliVerband*-Software als umfassender Branchenlösung für die Sportverbandsverwaltung.

1.2.2 Auftraggeber

Auftraggeber und interner Kunde ist der Geschäftsführer des Bereichs Software-Entwicklung bei der Xyz, Herr Reiffer. Als Teamleiter Web-Solutions nimmt Herr Zimmer die Rolle des Projekt- und Produktverantwortlichen für die *IntelliVerband*-Software insgesamt wahr.

1.2.3 Referenzkunden

Unabhängig von der generischen Anforderung stehen Referenzkunden aus dem Sportverbandswesen für die konkrete Anforderungsanalyse und Konzeptvalidierung sowie mit Realdaten für Import-Tests zur Verfügung.

1.3 Projektziele

Die Bestandsmeldung der Sportvereine wird jährlich vom *Deutschen Olympischen Sportbund* (DOSB) durch seine Landesmitgliedsorganisationen über unterschiedliche Systeme erhoben.

Die zu entwickelnde Importschnittstelle soll die über externe Systeme erhobenen Daten in das Modul "Bestandsdaten" der *IntelliVerband*-Software einlesen, mit den vorhandenen Organisationsdaten verknüpfen, sowie für die Anzeige in Berichten im Portal transformieren und aufbereiten.

1.4 Ist-Zustand

Für die jährliche Bestandsmeldung der Sportvereine an die Landessportorganisationen existieren je nach Bundesland unterschiedliche Systeme diverser Anbieter. Mit dem *IntelliVerband*-System bietet auch die Xyz einen eigenen online-gestützten Prozess für die Bestandsmeldung gemäß den Vorgaben des DOSB an.

Die über *IntelliVerband* erhobenen Daten werden im Modul 'Bestandsdaten' in entsprechende Tabellen jahrgangsweise gespeichert und stehen in Form von unterschiedlichen Berichten, nach diversen Dimensionen und in aggregierter Form zur Verfügung. Eine Übertragungsmöglichkeit von Bestandsmeldungen aus anderen Systemen auf direktem Wege besteht nicht.

1.5 Soll-Zustand:

Daten aus Bestandsmeldungen, die über die Systeme von Drittanbietern erhoben werden, sollen für Xyz-Kunden über einen geeigneten Weg der Datenmigration im eigenen Portal verfügbar gemacht werden. Aufgrund der Vielzahl unterschiedlicher Herkunftssysteme ist hierfür ein möglichst generischer Ansatz unter Nutzung üblicher Standards gewünscht.

Ziel ist es, denjenigen Kunden, die vertraglich noch an die Nutzung von Drittanbietersystemen für die Bestandserhebung gebunden sind, einen optimalen Nutzen Ihres *IntelliVerband*-Portals zu bieten. Dies soll durch die Integration und Nutzung der auf externem Wege erhobenen Daten in *IntelliVerband* gewährleistet werden.

Das folgende Use-Case-Diagramm (Abb. 1) zeigt den Gesamtkontext der Datenmigration: Von der Online-Bestandsmeldung im Drittanbieter-System durch die Vereine (grün) bis zum Import der Daten in das *IntelliVerband*-System durch einen Support-Mitarbeiter der Xyz (blau). Der Sportbund (schwarz) als Nutzer beider Systeme steht in der Mitte und hat einen Zusatznutzen seines *IntelliVerband*-Portals durch Integration der Daten.

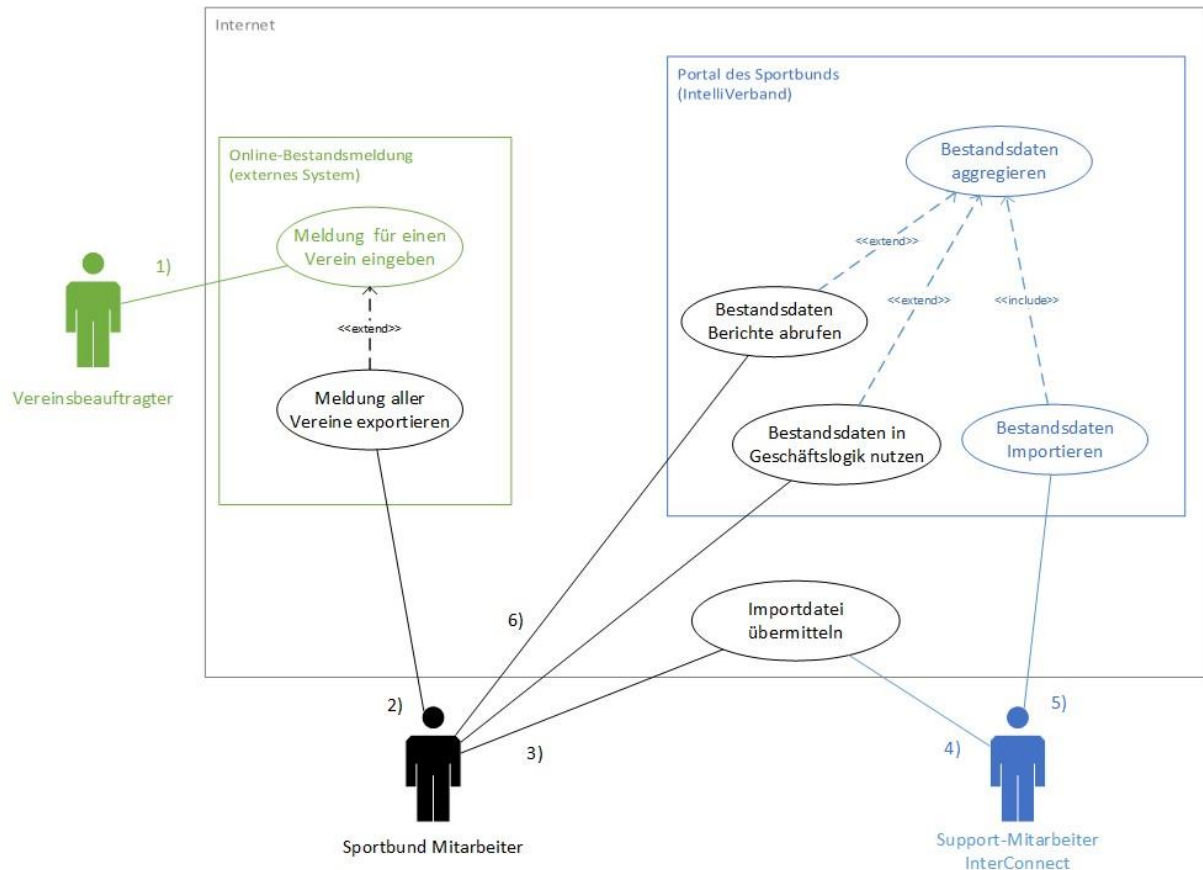


Abb. 1: Bestandsmeldung, Dateitransfer und Verarbeitung als Gesamtprozess.

1.6 Teilaufgaben

1.6.1 Erstellung Importformatvorlage

Zur Erreichung des Projektziels soll zuerst ein Datei- und Datenaustauschformat definiert werden, welches von Drittanbietersystemen ohne großen Aufwand als Datenexport erzeugt werden kann. Das Ergebnis soll eine Importformatvorlage für die Datenmigration sein.

1.6.2 Programmierung Import

Für die Datenübernahme in das *IntelliVerband*-System ist im nächsten Schritt eine Importroutine zu implementieren, die die Daten in einen Zwischenspeicher einliest. Nach erfolgter Plausibilitätsprüfung und Ermittlung der Fremdschlüssel aus den Stammdaten des Kernmoduls, sollen die Daten dann in die Tabellen des Bestandsmoduls geschrieben werden. Eine Protokollfunktion soll Erfolg und Misserfolg der Operationen dokumentieren.

Hierbei notwendige Datentransformationen in das Schema der Zieltabellen der Datenbank sind zu berücksichtigen und entsprechende SQL-Kommandos bzw. deren programmatisch gesteuerter Aufruf in der Geschäftslogik zu realisieren. Dabei sind Softwarearchitektur- und Design-Vorgaben von Xyz zu beachten.

1.6.3 Datenaggregation

Im letzten Schritt soll nach erfolgreichem Einlesen der jahrgangsweisen Bestandsmeldungen auch eine Aggregation der Zahlen über vorgegebene Altersgruppen erfolgen. Diese werden in separaten Tabellen in der Datenbank gespeichert zur Darstellung in Berichten.

2. RESSOURCEN UND ABLAUFPLAN

2.1 Personalplanung

Die Projektdurchführung wurde mit einer Person geplant, die für die gesamte Umsetzung – von der Analyse und Konzeption über die Implementierung und Qualitätssicherung bis zur Dokumentation – verantwortlich zeichnet. Projektbearbeiter war Markus Prolyingheuer.

Zur Validierung und Abnahme wurden außerdem Rückkopplungsschleifen mit den internen Projektauftraggebern und einem externen Pilotkunden vorgesehen.

Projektverantwortlicher im Betrieb war der Teamleiter, Herr Z. Projektauftraggeber war Herr X. (Leiter Web-Solutions und Gesellschafter der *Xyz GmbH & Co. KG*). Teamintern stand Frau Y. als erfahrene IntelliVerband-Entwicklerin zur Seite.

Als Pilotkunde war der *Landessportbund Hessen* (LSBH) mit den Herren Müller und Meier für Feedback zum Importformat und zur Bereitstellung von realen Testdaten mit eingebunden.

Eine darüber hinaus gehende Verteilung von Projekt(teil)aufgaben auf weitere Personen, versprach hinsichtlich Fertigstellungstermin und Qualität keinen Zusatznutzen. Im Gegenteil war davon auszugehen, dass sich der Abstimmungsaufwand erhöht, während ein Zeitgewinn durch Parallelisierung von Arbeitsschritten nur in geringem Maße zu erwarten wäre.

Lediglich für umfangreichere Testszenarien, wie sie bei komplexeren Projekten üblich sind, wäre das Hinzuziehen einer weiteren Person als unabhängigen Tester geboten gewesen. Anstelle dessen wurde in diesem Projekt eine *Code-review* im Rahmen der Präsentation des Programms vor Auftraggebern und Kollegen der Abteilung als ausreichend erachtet.

2.2 Sachmittelplanung

Es wurde kein Extraaufwand im Bereich der Sachmittel für die Durchführung des Projekts geplant. Für die Umsetzung stand dem Mitarbeiter sein regulärer, voll ausgestatteter Arbeitsplatz mit PC mit Windows 8.1, Entwicklungsumgebung (Visual Studio 2012 und SQL Server 2012 Management Studio) und Office 2013 sowie ein 24" Monitor zur Verfügung.

2.3 Kostenplanung

Die Kostenplanung beinhaltet Kosten für Sachmittel und Personal.

Bei externer Beauftragung wird für Entwicklungsarbeiten an den *IntelliVerband*-Modulen ein Kostensatz von 100,- € pro Entwicklerstunde zu Grunde gelegt. Dieser deckt neben den reinen Personalkosten auch die Kosten für den PC-Arbeitsplatz einschließlich Stromverbrauch und Nutzung der betrieblichen Räumlichkeiten ab.

Bei einem Projektumfang von 70 Stunden ergeben sich somit **Projektkosten** in Höhe von insgesamt **7.000,- Euro**.

Bei separater Ermittlung des Sachkostenanteils wären die nachfolgend aufgeführten Sachmittel mit ihren anteiligen kalkulatorischen Kosten zu berücksichtigen. Wobei auf Grund des Microsoft Gold Partner Status der Xyz für Microsoft Software-Lizenzen keine extra Kosten anfallen, da diese im Rahmen der Mitgliedschaft im Microsoft Partnernetzwerks durch den Mitgliedsbeitrag abgegolten sind. Als Berechnungsbasis für zeitabhängige Kosten bzw. Nutzungsdauern wurde von 70 Projektstunden bzw. einem Monat ausgegangen.

Sachmittel	Kosten	Berechnungsbasis
Monitor BenQ 24" – Leasingrate	3,26 €	3,26 €/Mon.
Computer HP ProDesk – Leasingrate	14,91 €	14,91 €/Mon.
Visual Studio Professional 2012 – Lizenz	- €	kostenlose Lizenzen im Rahmen der Mitgliedschaft im Microsoft Partnernetzwerk
SQL Server Management Studio – Lizenz	- €	
Microsoft Office Professional 2013 – Lizenz	- €	
Stromverbrauch Computer HP ProDesk 600 G1	4,93 €	320 W, 70 h, 0,22 €/kWh
Stromverbrauch Monitor BenQ GL2460HM 24"	0,42 €	27 W, 70 h, 0,22 €/kWh
Gesamt	23,51 €	

Tab. 1: Sachmittelkosten kalkulatorisch.

2.4 Terminplanung

Als Terminrahmen für die Projektdurchführung war der Zeitraum ab Projektgenehmigung am 14. September 2015 bis spätestens 15. November 2015 gesetzt.

Für die Terminplanung waren Tage mit Blockunterricht beim Bildungsträger („Finish-IT“ Lehrgang bei Lutz & Grub) auszusparen sowie der Urlaub des Projektauftraggebers zu berücksichtigen. Die gesetzlichen Feiertage im Durchführungszeitraum (Tag der Deutschen Einheit, Allerheiligen) fielen dagegen jeweils auf ein Wochenende, so dass diese für die Terminplanung keine Rolle spielten.

Die konkrete zeitliche Planung sah schließlich vor, dass werktäglich ca. 50% der Arbeitszeit für Supporttätigkeiten im Tagesgeschäft reserviert blieben, und somit für die Projektarbeit pro Arbeitstag jeweils rund 4 Stunden zur Verfügung standen.

2.5 Ablaufplan

Der Projektablauf gliederte sich in vier Phasen:

- 1) Analyse und Planung
- 2) Durchführung und Implementierung
- 3) Qualitätskontrolle und Abnahme
- 4) Projekt- und Anwenderdokumentation

Ein Gantt-Diagramm mit der konkreten Zeit-/Ablaufplanung findet sich im Anhang A (Abb. 10).

3. DURCHFÜHRUNG

3.1 Analyse und Konzeption

3.1.1 SOLL-/IST-Analyse: Vorgaben des DOSB

In der Anfangsphase des Projekts wurden Informationen gesammelt: So wurden als erstes auf der Website des Deutschen Olympischen Sportbunds (www.dosb.de) Vorgaben und Hintergründe zur jahrgangsweisen Bestandsdatenerhebung recherchiert.

Nachfolgendes Zitat aus einer über die DOSB-Website abrufbaren Broschüre [1] gibt den Grundsatzbeschluss wieder, der für alle Mitgliedsorganisationen bindend ist:

"In den Bestandserhebungen der Landessportbünde (A-Zahlen) sind alle Vereinsmitglieder – nach Geschlechtern getrennt – jahrgangsweise zu erfassen. Eine Unterscheidung in 'aktive' und 'passive' Mitglieder bzw. zeitlich befristete Mitgliedschaften erfolgt nicht." [1]

Schlüsselbegriff ist hier "jahrgangsweise", da dies eine Abkehr vom zuvor praktizierten System der nach Altersgruppen zu meldenden Mitgliederzahlen darstellt. In der selben Quelle zur Bestandserhebung wird zudem ausgeführt, dass jedes Vereinsmitglied einem Fachverband zu zuordnen ist, entsprechend der im Verein betriebenen Sportart – die sogenannten B-Zahlen. Dabei ist festgelegt, dass eine Sportart genau einem Sportfachverband im DOSB zugeordnet ist. Entsprechende Zuordnungslisten von Sportarten zu Fachverbänden lassen sich auf der DOSB-Website ebenfalls herunterladen.

Des Weiteren wurde auf der DOSB-Website eine Referenzliste [2] von Software-Systemen gefunden und studiert, die die DOSB-Schnittstelle zur Bestandsdatenerhebung unterstützen. Die Idee, diese Liste als Ausgangspunkt für Recherchen zu technischen Spezifikationen von Exportformaten zu nutzen, wurde schnell als zu zeitaufwendig verworfen – aufgrund der Vielzahl unterschiedlicher Quellsysteme und schlechter Zugänglichkeit der Information.

3.1.2 IST-/SOLL-Analyse: IntelliVerband-System

Im nächsten Schritt folgte eine eingehendere Analyse des *IntelliVerband*-Systems. Zum einen unter dem Gesichtspunkt, dass *IntelliVerband* selbst als Quellsystem von Bestandsdatenmeldungen fungieren kann. Zum anderen wurden hinsichtlich des Imports die Datenbanktabellen und Datentypen des Moduls *Bestandserhebung* analysiert sowie Querverbindungen zu anderen Modulen und Datenbanktabellen (Fremdschlüssel-Beziehungen) identifiziert. Ein Ausschnitt aus dem Datenmodell findet sich hierzu im Anhang B (Abb. 11).

Aus den gesammelten Informationen wurden die notwendigen Datenfelder für den Import der Mitgliederbestandsdaten ermittelt und daraus die Datentypen und Pflichtfelder für das neue Importtabellenformat bestimmt. Auch das Bericht-Layout der im *IntelliVerband*-Portal selbst angebotenen Exporte mit jahrgangsweise aufgeschlüsselten Mitgliederzahlen wurde gesichtet und für die Konzeption des Dateiaufbaus für das Importformat berücksichtigt.

3.1.3 Konzeption Importdateiformat

Ergebnis der Analyse war ein erster Entwurf in Gestalt einer Excel-Tabelle: Ein Tabellenformat zur Erfassung eines Vereins und dessen Mitglieder pro Sportart bzw. Fachverband in einer Zeile, mit den Anzahlen pro Geschlecht für jeden Jahrgang (von 0 bis 110 Jahren, bezogen auf das jeweilige Erhebungsjahr). Das bedeutete eine sehr breite Tabelle mit über 220 Spalten (vgl. dreiteilige Abb. 2 auf der folgenden Seite). Die obere Grenze von 110 Jahren orientiert sich hierbei am höchsten Lebensalter, was Menschen in Deutschland derzeit erreichen. Diese Festlegung wurde in *IntelliVerband* allerdings schon bei der Erstentwicklung des Bestandsmoduls getroffen und für dieses Projekt als gegeben übernommen.

	A	B	C	D	E	F	G	H
1	Nummer	ErhebungStichtag	Fachverband	DatumEingang	DatumErfassung	DatumKomplett	DatumPlausibel	MemoMeldung
2	1033	01.01.2014	0	20.02.2014	28.02.2014	28.02.2014	28.02.2014	
3	1033	01.01.2014	0065	20.02.2014	28.02.2014	28.02.2014	28.02.2014	
4	1040	01.01.2014	0	15.03.2014	31.03.2014	01.04.2014	01.04.2014	Hinweistext zu dieser Meldung
5	1040	01.01.2014	0030	15.03.2014	31.03.2014	01.04.2014	01.04.2014	
6	1040	01.01.2014	0073	15.03.2014	31.03.2014	01.04.2014	01.04.2014	
7								

	I	J	K	L	M	N	O	P
1	AnzahlAlter0M	AnzahlAlter0W	AnzahlAlter1M	AnzahlAlter1W	AnzahlAlter2M	AnzahlAlter2W	AnzahlAlter3M	AnzahlAlter3W
2	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0
4	1	0	0	2	2	4	1	2
5	0	0	0	0	0	0	0	0
6	0	0	0	0	0	0	0	0
7								

	HP	HQ	HR	HS	HT	HU	HV
1	AnzahlAlter107W	AnzahlAlter108M	AnzahlAlter108W	AnzahlAlter109M	AnzahlAlter109W	AnzahlAlter110M	AnzahlAlter110W
2	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0
6	0	0	0	0	0	0	0
7							

Abb. 2 a-c: Dateivorlage für Import mit Jahrgängen in 220 Spalten (Ausschnitt).

3.1.4 Abnahme Dateivorlage durch internen Auftraggeber

Dieses Importformat wurde dem Projektverantwortlichen Werner Zimmer vorgestellt, da er die Entwicklung der *IntelliVerband*-Software insgesamt koordiniert und die Anforderungen aus langjährigen Kundenbeziehungen und eigener Sportvereinsverwaltungspraxis sehr gut kennt.

Die Auswahl der Spalten und Datenformate wurden besprochen, sowie welche Felder als Pflichtfelder behandelt werden müssen, da sie in der Datenbank als NOT NULL definiert sind.

Die große Spaltenanzahl für alle potentiell vorkommenden Altersjahrgänge bis 110 Jahre und damit einhergehenden vielen überflüssigen Datenfeldern mit dem Wert "0" wurde diskutiert. Schließlich wurde das vorgelegte Format (mit einem Datensatz pro Erhebung, Verein und Fachverband in einer Zeile) ohne wesentliche Änderung als Importdateivorlage vom Projektverantwortlichen abgenommen.

3.1.5 Validierung durch externen Kunden

Zur frühzeitigen Validierung in der Praxis wurde beschlossen, das Importformat einem Kunden zwecks Stellungnahme zur Verfügung zu stellen. Dazu wurde die Dateivorlage mit einigen Beispieldatensätzen befüllt sowie notwendige Hinweise für den Kunden in einem Merkblatt "Importformat Erläuterungen" zusammengestellt (vgl. Anhang C. Anwenderdokumentation, Abschnitt 6.2).

Vom Testkunden kam dann die Rückmeldung, dass ihm der Export seiner Daten in einem anderen Format leichter fiele, wo pro Zeile die Mitgliederzahlen (männlich/weiblich) für nur einen Jahrgang (pro Verein, pro Fachverband) ausgegeben werden.

3.1.6 Re-Analyse und Änderung Dateiformat

Die Rückmeldung des Kunden wurde auf Auswirkungen für die Importdatei und die darauf anzuwendende Importroutine hin analysiert.

Es wurde grundsätzlich festgestellt, dass das gewünschte Format näher am Schema der Zieltabelle 'BstZahlenJahrgang' in der IntelliVerband-Datenbank ist, was die Verarbeitung während des Imports erleichtern würde – vgl. Datenbankschema im Anhang B (Abb. 11).

Die Importdatei würde dabei um viele Spalten schlanker werden, allerdings auch deutlich mehr Datensätze (→ Jahrgänge in Zeilen) umfassen. Zudem würde sich die Redundanz bei den Informationen zur Erhebung und der Verein-Fachverband-Zuordnung erhöhen. Letztendlich wäre die Importdatei aber auch für andere Kunden leichter zu generieren.

Da die Vorteile überwogen und noch kein Entwicklungsaufwand entstanden war, wurde dem Änderungswunsch entsprochen: Das Tabellenlayout der Importdatei wurde angepasst, wie in nachfolgender Abbildung (Abb. 3) gezeigt, und auch die "Importformat Erläuterungen" entsprechend neu gefasst.

	A	B	C	D	E	F	G	H	I	J	K
1	Nummer	ErhebungStichtag	Fachverband	DatumEingang	DatumErfassung	DatumKomplett	DatumPlausibel	MemoMeldung	Jahrgang	AnzahlM	AnzahlW
2	1033	01.01.2014	0	20.02.2014	28.02.2014	28.02.2014	28.02.2014		1984	2	0
3	1033	01.01.2014	0065	20.02.2014	28.02.2014	28.02.2014	28.02.2014		1984	2	0
4	1033	01.01.2014	0	20.02.2014	28.02.2014	28.02.2014	28.02.2014		1995	0	1
5	1033	01.01.2014	0065	20.02.2014	28.02.2014	28.02.2014	28.02.2014		1995	0	1
6	1040	01.01.2014	0	15.03.2014	31.03.2014	01.04.2014	01.04.2014	Hinweis zu dieser Vereinsmeldung	1957	1	3
7	1040	01.01.2014	0030	15.03.2014	31.03.2014	01.04.2014	01.04.2014	Hinweis zu dieser Vereinsmeldung	1957	1	1
8	1040	01.01.2014	0073	15.03.2014	31.03.2014	01.04.2014	01.04.2014	Hinweis zu dieser Vereinsmeldung	1957	1	2
9											

Abb. 3: Dateivorlage für den Import der Jahrgänge in Zeilen.

3.1.7 Detailanalyse Datenfluss und Tabellen

Für das neu festgelegte Importdateiformat wurde nun der Datenfluss analysiert (vgl. Abb. 4):

Zum einen wurden die Zieltabellen zum Schreiben der Importdaten in die Datenbank bestimmt. Zum anderen wurden die Tabellen für den lesenden Zugriff bestimmt – zur Ermittlung der Fremdschlüssel für Erhebung, Vereine (KrnOrganisation) und Fachverbände, auf welche beim Einfüge-Vorgang referenziert werden.

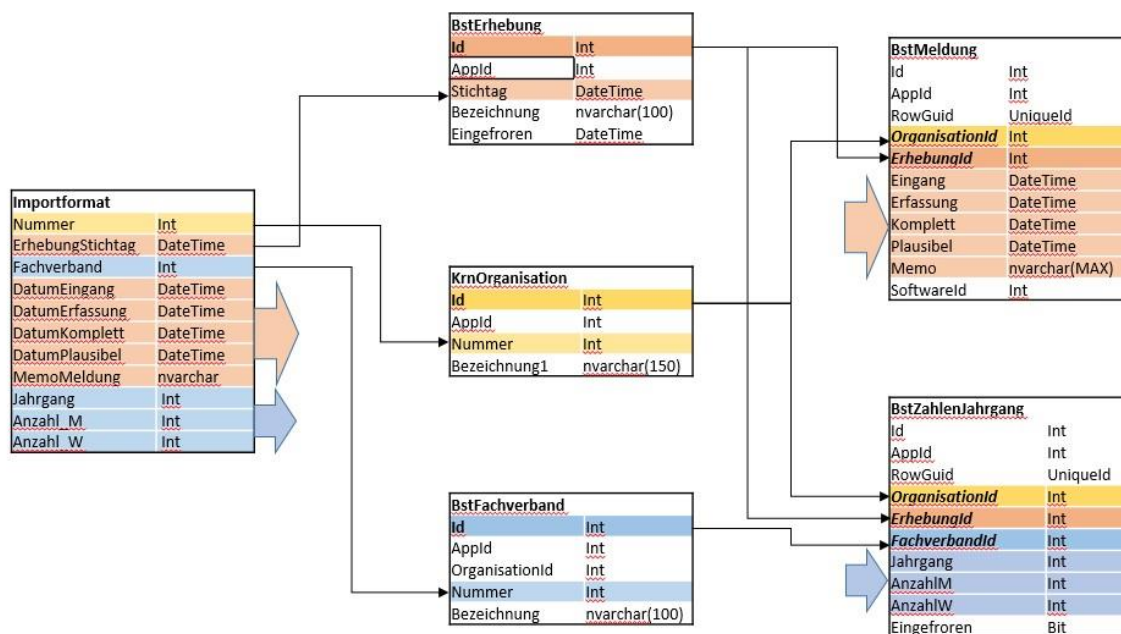


Abb. 4: Datenfluss: Importdatei, Fremdschlüsselermittlung und Zieltabellen.

3.1.8 Konzeption und Planung Programmablauf

Nach Ermittlung der betroffenen Tabellen und des Datenfluss insgesamt, wurde ein Lösungsplan entworfen und der Programmablauf im Groben skizziert:

1. Eine Importstarter-Applikation als Konsolenprogramm
2. Nutzung der Klassen der Geschäftslogik (BOL) und Erweiterung um Methoden für Import, Validierung und Datenbankoperationen (Lesend/Schreibend)
3. Weiterverarbeitung der importierten Daten für Berichte nach Altersgruppen

Für die Importstarter-Applikation wurde bewusst die Entscheidung für ein Konsolenprogramm getroffen, um nicht zu viel Aufwand in eine Benutzeroberfläche zu investieren.

Da die Migration von Bestandsmeldungsdaten keinen alltäglichen Geschäftsvorfall darstellt, wurde in Abstimmung mit dem Auftraggeber festgelegt, dass dies in der Praxis durch einen Support-Mitarbeiter der Xyz vorgenommen wird und nicht vom Kunden selbst.

Dementsprechend wurde die Lösung insgesamt für die Nutzung durch einen Entwickler in seiner Entwicklungsumgebung bzw. als Kommandozeilenprogramm zur Ausführung auf dem Produktiv-System konzipiert.

Für die Verarbeitung in der Geschäftslogik wurde der Programmablauf wie folgt umrissen:

- *LeseDateiInDatenbank()*
- *LeseDatenInDataSet()*
- *PrüfeDataSetAlleSpaltenVorhanden()*
- *PrüfeDataSetGültigerDatenTypInAllenFeldern()*
- *LeseDatenfeldMeldungUndPrüfeFremdschlüsselID()*
- *LeseDatenfeldVereinUndPrüfeFremdschlüsselID()*
- *LeseDatenfeldFachverbandUndPrüfeFremdschlüsselID()*
- *ErstelleDatensatzFürSchreibenInTabelleMeldung()*
- *ErstelleDatensatzFürSchreibenInTabelleZahlenJahrgang()*
- *LeseDatenAusTabelleZahlenJahrgang()*
- *AggregiereAnzahlZahlenJahrgang()*
- *SchreibeSummenInTabelleZahlenAltersgruppen()*

Die schrittweise Verfeinerung des Programmablaufs erfolgte im Rahmen der Implementierung.

3.2 Umsetzung und Implementierung

Für die Umsetzung der Programmierung stand ein Entwickler-PC mit Windows 8.1 Betriebssystem sowie installiertem Visual Studio Professional 2012 und Microsoft SQL Server 2012 Management Studio zur Verfügung.

Da die Programmierung der *IntelliVerband*-Software vor über 7 Jahren begonnen und seither kontinuierlich weiter entwickelt wurde, existieren bereits umfangreiche Klassenbibliotheken in über 10 Modulen. Durch diese Historie bedingt und aufgrund der Positionierung von Xyz als Microsoft-Partner, war die Entscheidung für die zu verwendende Technologie mit dem .NET-Framework vorgegeben. Da bisher nahezu ausschließlich in VB.NET entwickelt wurde, stand die Entscheidung für die zu nutzende Programmiersprache ebenfalls fest – um die Les- und Wartbarkeit des Codes für alle Entwickler zu erleichtern und Wiederverwertbarkeit zu gewährleisten.

3.2.1. Erstellung der Datei-Vorlage

Die Erstellung der Datei-Vorlage in Excel – gemäß eigener Phasenplanung der Umsetzung zugeordnet – fand zeitlich bereits während der Konzeptionsphase statt, wie unter 3.1.3 *Konzeption Importdateiformat* und 3.1.6 *Re-Analyse und Änderung Dateiformat* beschrieben.

Die Festlegung auf Microsoft Excel als Dateiformat und Programm zur Tabellenbearbeitung erfolgte aufgrund der weiten Verbreitung als Standardsoftware und hohen Akzeptanz in der Geschäftswelt allgemein. Die Nutzung von Excel auch in früheren Datenmigrationsprojekten (Import von Vereinsstammdaten ins Kernmodul) ließ zudem auf gute Erfahrungen und wiederverwertbaren Code zurückgreifen. Dazu bieten heutzutage die meisten datenbankbasierten Anwendungssysteme auch Excel-Exporte alternativ zum kommagetrennten CSV-Format an. Nicht zuletzt haben datentechnische Laien mit CSV-Dateien mitunter Probleme (speziell wenn das CSV-Dateiformat mit Excel als Anwendungsprogramm verknüpft ist und die AutoformatFunktion von Excel englisch punktgetrennte Dezimalzahlen und Datumswerte falsch darstellt).

Wegen der einfachen Handhabung des alten .xls-Formats im .NET-Umfeld sowie der Abwärtskompatibilität des moderneren .xlsx Formats, wurde für die Datei-Vorlage und Verarbeitung das alte Format gewählt. Da im Bereich der Non-Profit-Organisationen die Modernisierungszyklen für Software oftmals länger dauern, ist die Nutzung älterer Office-Pakete unter *IntelliVerband*-Kunden auch durchaus noch verbreitet.

Die Limitierung auf 256 Spalten und 65.536 Zeilen im Excel-Format 97–2003 gegenüber 16.384 Spalten und 1.048.576 Zeilen ab Excel 2007 und später, war bei der Entscheidung nochmals bewusst überlegt und für die Umsetzung in Kauf genommen worden.

3.2.2 Programmierung des Datei Imports

Die Programmierung des Datei-Imports erfolgte mit den einfach gehaltenen Möglichkeiten eines Konsolenprogramms als ImportStarter im Namespace: *IWeb.SportBiz.Krn.Import*.

Die SqlConnection für die genutzte Testdatenbank wurde direkt in die *app.config* eingetragen.

Neben der Klasse *System.IO* wurden fürs Projekt auch Hilfsklassen des eigenen Frameworks genutzt: *IWeb.Tools.AppTables.BusinessEntity* und *IWeb.Tools.AppTables.BusinessObject*

Ferner wurden zur Nutzung bestehender Klassen aus der Geschäftslogik von *IntelliVerband* (alias *SportBiz* – alter Projektname, der sich in den verwendeten Namespaces wiederfindet) die Namespaces *IWeb.SportBiz.Krn.BusinessObject* und *IWeb.SpOrtBiz.Bst.BusinessObject* eingebunden.

Beim Programmaufruf wird ein Kommando-Parameter mit Befehlsargument in der Konsole übergeben. Der Kommando-String wird in seine zwei Bestandteile (*cmd* und *arg*) zerlegt und mittels Fallunterscheidung entweder die Methode *InitJob()* oder *StartJobBstMeldung()* mit dem zweiten Teilstring als Argument ausgeführt (vgl. Abb. 5):

- *InitJob()* erwartet als Argument den Pfad und Dateinamen der Importdatei, liest die Datei binär als Image ein und schreibt Statusinformationen zum Job in die Datenbank (Tabelle AppJob).
- *StartJobBstMeldung()* erwartet als Argument die *JobId* des zuvor initiierten Jobs, um das darüber referenzierte Datei-Image im Bestandsdatenmodul weiter zu verarbeiten.

Die im weiteren Verlauf programmierten Methoden wurden dann in der Geschäftslogik der beteiligten Module verortet. Das ImportStarter-Programm ruft hier über eine entsprechende Objektinstanz der Klasse *BOLBstMeldung* die Methode *ImportBstMeldungJob()* auf.

```
Public Shared Sub Main(args As String())

    If args.Length = 1 Then
        Dim cmd As String = args(0).Substring(0, 3)
        Dim arg As String = args(0).Substring(3)

        Select Case cmd
            Case "/i:" ' Parameter = FileName
                InitImportJob(arg)
            Case "/o:" ' Parameter = JobId
                StartJobOrganisation(Integer.Parse(arg))
            Case "/b:" ' Parameter = JobId
                StartJobBstMeldung(Integer.Parse(arg))
            Case "/w:" ' Parameter = ImageId
                WriteImage(Integer.Parse(arg))
        End Select
    End If
```

Abb. 5: Main-Methode mit einfachem Select Case für Aufrufparameter.

3.2.3 Programmierung Datenverarbeitung Geschäftslogik

Der Hauptteil der Implementierung betraf nun die Verarbeitung der als Image gespeicherten Importdaten aus der Excel-Datei. Hierzu wurden die Binärdaten aus dem Image erst nochmal zurück in eine temporäre Datei geschrieben und diese mit Hilfe des OLEDB-Providers für Excel als XML-Repräsentation über einen *StringReader* in ein *DataSet* eingelesen.

Die Spaltennamen aus der ersten Zeile der Excel-Tabelle wurden dann ausgelesen, im Attribut *ColumnName* der *DataTable.DataColumn* gespeichert und die erste Zeile aus der *DataTable* entfernt. Darauf folgte die Prüfung, ob alle vom Importformat definierten Spalten in der Datei vorhanden sind und eingelesen wurden: *CheckIfTableContainsAllMandatoryColumns()*

Bei negativer Prüfung sollte ein Eintrag ins *AppLog* (= Tabelle in der *IntelliVerband*-Datenbank) geschrieben sowie eine *Exception* geworfen und der Importvorgang abgebrochen werden.

Bei Vorhandensein der vorgegebenen Spalten folgten zwei weitere Prüfungen: Zum einen musste sichergestellt werden, dass die Werte in den Spalten auch dem erwarteten Datentyp entsprachen und die Felder einen plausiblen Wert aufwiesen (z.B. Zahlen nicht negativ). Die zweite Prüfung betraf die Spalten mit den Fremdschlüsselverweisen: Hier musste für die 'Erhebung', die 'Nummer' (Verein) und den 'Fachverband' in den Basistabellen das Vorhandensein der referenzierten Entität geprüft und die Id für die Fremdschlüsselbeziehung ermittelt werden. Auch hier sollte im Fehlerfall eine Nachricht ins *AppLog* geschrieben, eine *Exception* geworfen und der Importvorgang abgebrochen werden.

Hinsichtlich der drei Fremdschlüsselfelder wurde überlegt, ob ggfs. durch den BestandsdatenImport auch neue Einträge in den Basistabellen angelegt werden könnten. Für den Verein und Fachverband wurde dies eindeutig verneint, mangels notwendiger Zusatzinformationen. Für die Erhebung, die sich primär durch das Datum des Stichtags definiert, hätte man es erwägen können. Allerdings wurde auch dies verworfen, um nicht auf

Basis von Zahlendrehern unkontrolliert und ungewollt neue Erhebungen anzulegen, die in der Realität nicht existierten.

Der die zwei Prüfungen realisierende Programmteil war in der ersten Fassung nicht ideal programmiert: Die *DataTable* wurde zeilenweise durchlaufen und in jeder Zeile wurden in 8 der 11 Spalten redundante Werte viele Male, immer wieder neu auf ihren Datentyp und Fremdschlüssel-ID geprüft. Hier musste eine effizientere Lösung überlegt werden.

Die Optimierung bestand darin, die beiden Prüfungen nicht auf der *DataTable* in jeder Zeile wiederholt für jede Spalte auszuführen. Sondern es sollten die Prüfungen pro Spalte separat nur auf die distinkten Wertausprägungen angewendet werden – redundante Werte eliminiert.

Hierzu wurde die *DataTable* in eine (sortierte) *DataView* überführt und diese den Methoden *CheckImportTableValues()* und *CheckImportForeignKeys()* jeweils als Argument übergeben.

Mit der Funktion *DataView.ToTable(distinct, Column)* konnten nun wiederum aus der *DataView* einspaltige Tabellen mit den eindeutigen Feldwerten erstellt werden und diese jeweils in eigenen Schleifen durchlaufen und effizient geprüft werden (vgl. Code-Ausschnitt in Abb. 6) .

```
Private Sub CheckImportTableValues(dv As DataView, message As StringBuilder)
    Dim tmpTable As New DataTable()
    Dim msgFormat = "Fehler in Spalte {1}: {2}{0}"
    Dim parseDateTime As DateTime

    ' Überprüfung der Zahlenfelder: Nummer, Fachverband, Jahrgang, AnzahlM, AnzahlW

    tmpTable = dv.ToTable(True, "Nummer")
    For Each row As DataRow In tmpTable.Rows
        If Not ((IsNumeric(row("Nummer").ToString())) _
            AndAlso (CType(row("Nummer").ToString(), Integer) > 0)) Then
            message.Append(String.Format(msgFormat, vbCrLf, "Nummer", _
                "Zahl erwartet. Feldinhalt kann nicht verarbeitet werden: " _
                & row("Nummer").ToString()))
        End If
    Next
```

Abb. 6: Code zur Prüfung der Wertausprägungen in Spalte "Nummer" (Redundanz eliminiert).

In ähnlicher Weise wie oben für die Methode *CheckImportTableValues()* gezeigt, wurde auch bei der Prüfung der Fremdschlüssel in der Methode *CheckImportForeignKeys()* verfahren.

Wobei hier im Unterschied zu oben mit der Funktion *DataView.ToTable(distinct, Columns)* alle drei interessierenden Spalten der temporären Tabelle übergeben wurden. Dadurch bezieht sich die Eigenschaft "distinct" hier auf die Kombination der Ausprägungen von 'Erhebung', 'Nummer' und 'Fachverband'. Dieses Vorgehen wurde an der Stelle gewählt, um im Fehlerfall eine präzisere Beschreibung des Fehlerkontexts ins *AppLog* schreiben zu können. Da eine

fehlerhafte Erhebung oder Nummer bzw. die Fachverbandszuordnung stets im Zusammenhang einer konkreten Meldung eines Vereins (Nummer) zu betrachten ist.

Eine zweite Besonderheit dieser Methode ist, dass hier für die Interaktion mit der Datenbank nun mehrfach bereits existierende Funktionen der Geschäftslogik für den Datenzugriff genutzt werden konnten. Hierzu erfolgte zu Beginn ein größerer Block mit Deklaration / Initialisierung von *BOL*- und *DataRow*-Objekten aus Klassen des eigenen Frameworks (vgl. Abb. 7).

Die *BOL*-Klassen (*BOLBstErhebung*, *BOLKrnOrganisation*, *BOLBstFachverband*) repräsentieren dabei auf Geschäftsprozessebene die drei entsprechenden Tabellen in der Datenbank (*BstErhebung*, *KrnOrganisation*, *BstFachverband*) als Objekte mit geschäftsspezifischen Methoden, die wiederum über Klassen und Methoden der "Data Access Layer" (DAL) mit der Datenbank kommunizieren.

Eine zentrale Methode im folgenden Codeausschnitt ist dabei in jeder Klasse die Funktion *GetSingleRowWhere()*, hinter der sich ein SQL SELECT mit WHERE-Bedingung und mit genau einem Datensatz als *ResultSet* verbirgt. Das entsprechende *DataRow*-Objekt ist hier ebenfalls passend zum abgefragten Tabellenschema definiert, um das Ergebnis-Tupel in einer typisierten Datenstruktur aufzunehmen.


```
Private Sub CheckImportForeignKeys(dv As DataView, message As StringBuilder)
    Dim tmpTable As New DataTable()
    Dim msgFormat = "Fehler bei Verein Nummer {1}: {2}{0}"
    Dim parseDateTime As DateTime

    Dim bolErh As New BOLBstErhebung()
    Dim bolOrg As New BOLKrnOrganisation()
    Dim bolFvb As New BOLBstFachverband()

    Dim drErh As DSBstErhebung.BstErhebungRow = Nothing
    Dim drOrg As DSKrnOrganisation.KrnOrganisationRow = Nothing
    Dim drFvb As DSBstFachverband.BstFachverbandRow = Nothing

    tmpTable = dv.ToTable(True, "Nummer", "ErhebungStichtag", "Fachverband")

    For Each row As DataRow In tmpTable.Rows

        'Prüfung: Referenz (FK) auf Erhebung in BstErhebung -- Datumsformatierung beachten!
        If DateTime.TryParse(row("ErhebungStichtag").ToString(), parseDateTime) Then
            drErh = bolErh.GetSingleRowWhere(String.Format("Stichtag = " & _
                "convert(datetime, '{0}', 104)", parseDateTime))
        End If
        If drErh Is Nothing Then
            message.Append(String.Format(msgFormat, vbCrLf, row("Nummer").ToString(), _
                "Erhebung mit diesem Datum nicht bekannt: " & _
                & row("ErhebungStichtag").ToString()))
        End If

        ' Prüfung: Referenz (FK) auf Verein in KrnOrganisation
        If IsNumeric(row("Nummer").ToString()) Then
            drOrg = bolOrg.GetSingleRowWhere(String.Format("Nummer = {0}", row("Nummer")))
        End If
        If drOrg Is Nothing Then
            message.Append(String.Format(msgFormat, vbCrLf, row("Nummer").ToString(), _
                "Verein mit dieser Nummer nicht bekannt: " & _
                & row("Nummer").ToString()))
        End If

        ' Prüfung: Referenz (FK) auf Fachverband in BstFachverband
        If (IsNumeric(row("Fachverband").ToString()) _
            AndAlso CType(row("Fachverband").ToString(), Integer) > 0) Then
            drFvb = bolFvb.GetSingleRowWhere(String.Format("NummerWlsb = {0}", row("Fachverband")))
        End If
        If drFvb Is Nothing Then
            ' Fallunterscheidung: Feld leer bzw. 0 dann -> A-Zahl.
            '                               Nummer > 0 dann B-Zahl Fachverband)
            If Not ((row("Fachverband").ToString() = String.Empty) _
                Or (CType(row("Fachverband").ToString(), Integer) = 0)) Then
                message.Append(String.Format(msgFormat, vbCrLf, row("Fachverband").ToString(), _
                    "Fachverband mit dieser Nummer nicht bekannt: " & _
                    & row("Fachverband").ToString()))
            End If
        End If
    Next
End Sub
```

Abb. 7: Methode zur Prüfung der Fremdschlüsselreferenzen.

Ist dieser Programmabschnitt erfolgreich durchlaufen, kann schließlich die Vorbereitung der eingelesenen Daten zum Schreiben in die Datenbank erfolgen und ausgeführt werden.

Zur Implementierung des weiteren Ablaufs wurde nun wieder auf die zuvor genutzte, sortierte *DataView* zurückgegriffen. Der Vorteil einer *DataView* gegenüber der *DataTable* besteht in der Möglichkeit zur Sortierung nach festgelegten Sortierkriterien. Da die Daten der Excel-Tabelle inhaltliche Abhängigkeiten aufweisen und sich zudem unterschiedlich häufig ändern, erfolgte eine Sortierung der Spalten wie in Abb. 8 gezeigt.

```
' Sortierte DataView  
Dim dv As New DataView(ds.Tables(0))  
dv.Sort = "ErhebungStichtag DESC, Nummer ASC, Fachverband ASC, Jahrgang DESC"
```

Abb. 8: Überführung der DataTable in sortierte DataView.

Diese explizite Sortierung wurde gewählt, um damit eine bessere Kontrolle über die Steuerung des Programmflusses zu erreichen. Dabei lag die Annahme zu Grunde, dass im Normalfall meist nur eine Erhebung pro Datei importiert würde; d.h. die Referenz auf die Erhebung (über Spalte 'ErhebungStichtag') sich nur selten ändert. Pro Erhebung und Verein muss dann jeweils die Anlage einer Meldung in der Datenbank erfolgen, weswegen der Verein (Spalte: 'Nummer') im zweiten Block sortiert wurde. Innerhalb des Vereins erfolgte dann die Sortierung nach 'Fachverband', um schließlich pro Fachverband die Mitgliederanzahlen fortlaufend für alle Jahrgänge zu lesen und in die Datenbank zu schreiben.

Implementiert wurde dies mit einer *For-each*-Schleife über alle Datensätze, wobei die Änderungszustände der drei Dimensionen Erhebung, Verein und Fachverband durch die Nutzung entsprechender Hilfsvariablen (*[Dim]hasChanged as Boolean*) mitgeführt wurden. Jeweils nur wenn sich in einer der drei Dimensionen der Feldwert änderte, wurde für diesen die Fremdschlüssel-ID aus der Datenbank ermittelt. Hatten sich entweder die *OrganisationId* oder die *ErhebungId* geändert wurde eine neue Meldung angelegt. Ansonsten wurde mit jedem Schleifendurchlauf für die Mitgliederanzahlen jedes Jahrgangs ein neuer Datensatz (mit Verein und Fachverbandzuordnung) in die Tabelle *BstZahlenJahrgang* eingefügt.

Bei den einzelnen datenverarbeitenden Schritten wurden wiederum intensiv die Methoden der Framework-eigenen BOL-Klassen für Datenbankzugriffe sowie typisierte *DataRow*-Objekte für die Aufnahme der Datensätze genutzt.

Die gesamte Schleife mit allen Datenbankzugriffen – vom Lookup der Fremdschlüssel-IDs bis zur Anlage neuer Meldungen und dem Einfügen der Mitgliederzahlen in *BstZahlenJahrgang* – hatte dabei innerhalb einer Transaktion zu erfolgen und die Transaktion selbst war wiederum mit einem Try-Catch-Block abzusichern. Transaktionen helfen, Inkonsistenzen in der Datenbank zu vermeiden: So werden zusammengehörende Operationen auf einer Datenbank nur vollständig oder gar nicht durchgeführt und gegen gleichzeitigen Zugriff durch andere Prozesse geschützt (ACID-Prinzip). Im Fehlerfall oder bei Abbruch der Datenbankverbindung kann die Transaktion zurückgerollt und die Datenbank wieder in einen konsistenten Zustand versetzt werden.

Die Transaktion an dieser Stelle des Codes wurde deswegen sogar noch ausgeweitet auf die im folgenden Abschnitt beschriebene "Programmierung der Aggregation nach Altersgruppen": Nur wenn auch noch die Berechnung der Mitgliederzahlen entsprechend der Altersgruppen erfolgreich durchgeführt wurde, existieren konsistente Zahlen im System und allen Berichten.

3.2.4 Programmierung der Aggregation nach Altersgruppen

Über diesen Teil der Anforderung bestand während der Planungsphase zunächst wenig Klarheit bzgl. zeitlichen Aufwand und konkreter Implementierung. Mit fortschreitender Nutzung und Analyse der verfügbaren BOL-Klassen und ihrer Methoden, kristallisierte sich hierfür dann unerwartet große Unterstützung durch das Framework heraus: Da das *IntelliVerband*-System selbst die jahrgangsweise Bestandsdatenmeldung für Vereine über das sogenannte externe Portal ermöglicht und auch hier eine Aggregation der Mitgliederanzahlen nach Altersgruppen erfolgt, war von entsprechendem Programmcode im Bestandsdatenmodul auszugehen.

In der Tat existierte hier bereits die benötigte Methode, die aus der Tabelle *AppValues* die Altersgruppengrenzen ausliest, etwaig bestehende Zahlen für dasselbe Erhebungsjahr in der Tabelle *BstZahlen* löscht und sowohl innerhalb der Altersgruppen die Mitgliederanzahlen pro Verein und Fachverband aufsummiert als auch die Gesamtsummen männlich und weiblich berechnet: *BalancingBstZahlenJahrgang2BstZahlen()*

Diese Methode galt es nur mit einer Objektinstanz der Klasse *BOLBstZahlenJahrgang* und den entsprechenden Parametern aufzurufen und die Berechnungen auf der Datenbank ausführen zu lassen. Wichtig war an der Stelle, die Transaktionsunterstützung der Methode zu nutzen und auch diese Datenzugriffsoperationen in die eine große Transaktion einzubinden, die im vorherigen Abschnitt vor Beginn des ersten Schleifendurchlaufs gestartet und nach dieser Methode nun mit Commit (bzw. Rollback im Fall einer Exception) abgeschlossen wurde.

3.2.5 Integrationstest

Während der Importstart mit binären Einlesen der Datei als Image (und Ausgabe des selbigen wieder in eine Datei) noch isoliert getestet werden konnte, setzten alle folgenden Schritte das Vorhandensein von Importdaten voraus. So war auch schon frühzeitig die Anbindung einer Datenbank notwendig, um das Zusammenspiel von Programmierung und *Backend* zu testen. In diesem Sinne waren alle die Entwicklung begleitenden Tests bereits auch Integrationstests.

Als Feuertaufe wurde letztlich ein Testimport mit Realdaten eines Pilotkunden auf einer lokalen Kopie seiner Datenbank durchgeführt. Dabei konnte – ganz ohne vorsätzliche Manipulation – beim ersten Importversuch gleich ein Abbruch der Transaktion provoziert werden. So musste sehr realitätsnah im *AppLog* die Fehlermeldungen auf mögliche Ursachen hin analysiert und direkt behoben werden. Nach erfolgter Korrektur konnte dann auch die Transaktion erfolgreich abschließen und das Ergebnis in der Datenbank validiert werden: Sowohl hinsichtlich der einzelnen Datensätze im System (Tabelle *BstZahlenJahrgang*) als auch für die Summen der Altersgruppen konnte Übereinstimmung mit den Zahlen in der Excel Importdatei festgestellt werden. Der Integrationstest wurde somit erfolgreich bestanden.

3.2.6 Deployment

Ein explizites Deployment, im Sinne einer Software-Verteilung, fand nicht statt. Jedoch wurde der entwickelte Code regelmäßig über eine gemeinsam genutzte Quellcodeverwaltung den

Teammitgliedern (und Auftraggebern) auf dem *TeamFoundationServer* zur Verfügung gestellt. Von dort kann der Programmcode jederzeit aktuell abgerufen werden. Die eigenen in den BOL-Klassen ergänzten Methoden sind über die Projekt hinaus im System verfügbar.

3.3 Qualitätssicherung

3.3.1 Begleitende Tests

Von Beginn an wurden entwicklungsbegleitende Tests durchgeführt. Zum einen um die Funktionsweise von Methoden des Frameworks genau zu verstehen und optimal zu nutzen; zum anderen um zeitnah die Korrektheit und Performanz der eigenen Programmierung zu überprüfen und zu verbessern.

Beispielsweise war im Programmablauf die wiederholte Prüfung der vielfach redundanten Feldwerte in den Importdaten bei sequenzieller Verarbeitung mit dem klassischen *DataTables*Objekt nur unbefriedigend zu lösen. Hier Datenbankzugriffsoperationen einzusparen, barg offensichtlich einiges Potenzial. Wie im Abschnitt 3.2.3 *Programmierung Datenverarbeitung Geschäftslogik* bereits erläutert, wurde hier durch die alternative Nutzung eines *DataView*Objekts Verbesserungen in zweierlei Hinsicht erreicht:

Zum einen konnte durch die Sortierung der *DataView* eine bessere Kontrolle des Programmflusses erreicht werden, so dass nur noch bei Änderungen innerhalb einer Dimensionsspalte Datenbankabfragen zwecks Fremdschlüssel-Lookup abgesetzt wurden. Zum anderen konnte durch die Reduzierung der Spalten auf distinkte Werte – mittels der Funktion *DataView.ToTable(distinct, Column)* – deren Prüfung auf Typ-Kompatibilität und Plausibilität deutlich effizienter gestaltet werden.

Ein konkreter Vergleichstest für eine Importdatei mit 32.216 Datensätzen (264 Meldungen) ergab eine Verbesserung der Programmdurchlaufzeit um den Faktor 3.

Ergebnisse				Meldungen		
	JobId	Timestamp	Message	JobId	Dauer (Min.)	Bemerkung
1	17	2015-10-29 09:57:25.303	Beginn der Bearbeitung.	17	03:43,184	vor Optimierung
2	17	2015-10-29 10:01:08.487	Ende der erfolgreichen Bearbeitung.			
3	18	2015-10-29 13:21:24.647	Beginn der Bearbeitung.	18	01:11,120	nach Optimierung
4	18	2015-10-29 13:22:35.767	Ende der erfolgreichen Bearbeitung.			

Abb. 9: Verarbeitungsdauer von 2 Jobs vor und nach Code-Optimierung.

3.3.2 Test mit Pilotkunden

Zum Abschluss der Implementierung wurde, wie in Abschnitt 3.2.5 Integrationstest beschrieben, ein Integrationstest mit realen Daten eines Pilotkunden durchgeführt und diese in eine lokale Kopie seiner Datenbank importiert. Dieser Test verlief erfolgreich – sowohl hinsichtlich des Ergebnisses bei korrekter Verarbeitung, als auch bzgl. der Programmreaktion im Fehlerfall.

Über die Pivot-Funktion in Excel wurde auch die korrekte Summenbildung der Aggregation nach Altersgruppen überprüft – gegen die originalen Datensätze in der Importdatei wie auch gegenüber den jahrgangsweise eingelesenen Mitgliederzahlen.

3.3.3 Ergebniskontrolle

Die Ergebniskontrolle verlief in jeder Hinsicht zufriedenstellend. Die Tests im Kleinen – zur Optimierung der Performanz, wie oben dargestellt – wie auch der Integrationstest mit realen Importdaten und Datenbankumgebung waren erfolgreich: Transaktionen brachen ab, wenn sie fehlerbedingt zurückgesetzt werden mussten, bzw. terminierten mit dem gewünschten Ergebnis konsistenter Zahlen – sowohl in der jahrgangsweisen Betrachtung wie auch in den Berichten nach Altersgruppen. Nicht zuletzt ist auch eine operative Fehler-, Status- und Ergebniskontrolle über Einträge im *AppLog* möglich.

3.3.4 Abnahme durch Auftraggeber

Die Abnahme durch die Auftraggeber – dem Bereichsleiter und dem Projektverantwortlichen – erfolgte im Rahmen einer gemeinsamen Code Review, wo der Programmablauf besprochen und die Funktionsfähigkeit vorgeführt wurde. Die Ergebnisse kleinerer Optimierungen wurden dargestellt wie auch das Ergebnis des Integrationstests mit den realen Daten des Pilotkunden auf der lokalen Kopie der Datenbank präsentiert.

Damit wurde der Projektauftrag als erfolgreich umgesetzt bewertet und abgenommen.

3.3.5 Nacharbeiten

Nacharbeiten zum Ende der Implementierungsphase – zur Sicherstellung oder in Folge der Abnahme, wie bei der Planung ursprünglich angenommen – waren nicht notwendig. Stattdessen wurden schon beim Entwurf des Importformats, wie in Abschnitt 3.1.6 ausgeführt, auf Kundenwunsch hin ungeplante Nacharbeiten fällig. Im Sinne der kontinuierlichen Verbesserung wurden kleinere Nacharbeiten auch im Zuge der entwicklungsbegleitenden Tests durchgeführt. Beispielhaft hierfür sei die Umstellung von *DataTables* auf *DataViews* genannt, die eine effizientere Datenprüfung und bessere Kontrolle des Programmflusses ermöglichte.

3.4 Dokumentation

3.4.1 Anwenderdokumentation

Die Anwenderdokumentation richtet sich an Entwicklerkollegen im Team Web-Solutions der Xyz. *IntelliVerband*-Kunden werden nicht selbst Anwender des Programms sein. Im Gegenteil ist die Nutzung des Programms zur Bestandsdaten-Migration nur durch SupportMitarbeiter vorgesehen. Die Anwenderdokumentation dazu wurde im Rahmen des Projekts vom Projektbearbeiter erstellt und befindet sich im Anhang C. Anwenderdokumentation.

3.4.2 Kundendokumentation

Eine ausführliche Kundendokumentation gibt es nicht. Für den Kunden verfügbare Dokumente sind zum einen die Excel-Datei (*Importformat Bestandsmeldung Zeilen.xls*) mit einigen beispielhaften Datensätzen, sowie ergänzend dazu ein kompaktes Merkblatt als WordDokument (*Importformat Erläuterungen.docx*), wo Inhalt, Datentyp und Bedeutung der Felder kurz erläutert werden. Diese beiden Dokumente wurden während der

Konzeptionsphase vom Projektbearbeiter erstellt und in den Anhang der Anwenderdokumentation aufgenommen. Auffindbar – wie oben genannt – im Anhang C.

3.4.3 Projektdokumentation

Die Dokumentation der betrieblichen Projektarbeit wird durch das vorliegende Dokument erfüllt und wurde im Rahmen des Projekts teils entwicklungsbegleitend, größeren Teils aber erst während der Abschlussphase des Projekts, innerhalb des Gesamtprojektzeitraums vom Projektbearbeiter erstellt.

3.5 Abweichungen und Anpassungen

3.5.1 Anpassungen am Importdateiformat

Da die Realisierung der Schnittstelle im Zusammenhang mit einem anderen Kundenprojekt in absehbarer Zeit konkret gebraucht wird, wurde dieser externe Kunde zur Validierung des Importdateiformats und zur Lieferung realer Testdaten in die Entwicklung involviert. Bereits in der Konzeptionsphase, wurde dem Kunden der Erstentwurf des Importformats zur Verfügung gestellt und um Rückmeldung gebeten.

Vom Kunden wurde als Änderungswunsch formuliert, dass ein Export der Daten in einem anderen Format leichter fiele, wo pro Zeile die Mitgliederzahlen für nur einen Jahrgang (pro Verein und Fachverband) ausgegeben werden. Diese Rückmeldung des Kunden wurde alsdann auf Auswirkungen für die Importdatei und die zu programmierende Importroutine hin analysiert (vgl. Darstellung in Abschnitt 3.1.5 *Validierung durch externen Kunden* und 3.1.6 *Re-Analyse und Änderung Dateiformat*).

Dem Änderungswunsch wurde schließlich entsprochen und das Importdateiformat angepasst wie in Abb. 3 im Abschnitt 3.1.6 dargestellt.

Bei der Entscheidung wurde bewusst an der Nutzung des alten Excel-Formats (97-2003) festgehalten, trotz der bekannten Limitierung auf 65.536 Zeilen, die bei einem mittelgroßen Landessportverband durchaus schon erreicht werden können. Da ein Datei-Import ohnehin über einen Support-Mitarbeiter durchgeführt wird, ist hier situativ eine angemessene Lösung (z.B. durch Aufteilung auf mehrere Dateien) zu erwägen und anzuwenden.

Insgesamt überwogen die Vorteile – für den Kunden und bei der konkreten Datenverarbeitung: Die leichtere Generierbarkeit des gewünschten Tabellen-Layouts durch den Kunden, sowie die durchaus weite Verbreitung von .xls als Exportformat in älteren Quell-Systemen, waren ein Argument zur Beibehaltung der Abwärtskompatibilität. Ein weiteres Argument betraf die guten Erfahrungen mit den etablierten Verfahren und getesteten Methoden – Stichwort: Qualitätssicherung und Effizienz. Darüber hinaus wurde durch das abgewandelte Tabellenformat auch die Verarbeitung der jahrgangsbezogenen Anzahlen erleichtert: Die Anzahl der benötigten Spalten wurde reduziert und damit die Verarbeitung inhaltsleerer Datenfelder (mit Anzahl = 0) eliminiert. Aber auch Rechenaufwand für die Transformation von Jahrgangsalter (abhängig vom Erhebungsjahr) auf Geburtsjahrgänge wurde durch die Entscheidung eingespart.

3.5.2 Anpassungen der DataSets

Die Änderung der internen Datenhaltung durch Verwendung von *DataViews* anstelle von *DataTables* stellte zwar auch eine Abweichung gegenüber bisherigen Entwicklungsansätzen innerhalb der Software dar, war allerdings nicht als Abweichung der Implementierung gegenüber geplanten Funktionen oder Eigenschaften des Systems zu werten.

Weitere Abweichungen und Anpassungen gegenüber der Konzeption gab es nicht.

4. PROJEKTERGEBNIS

4.1 Soll-/Ist-Vergleich

Das entwickelte Importprogramm liest Mitgliederbestandsdaten aus externen Quellen in das Modul 'Bestandsdaten' der *IntelliVerband*-Software ein und stellt Fremdschlüsselbeziehungen mit den vorhandenen Organisationen und Fachverbänden her. Die Mitgliederzahlen werden jahrgangsweise abgespeichert und auch für Berichte gemäß der hinterlegten Altersgruppendefinitionen aufsummiert und zur Ansicht im Portal bereitgestellt (IST).

Damit sind die im Abschnitt 1.3 *Projektziele* formulierte SOLL vollumfänglich erreicht.

4.2 Abweichungen

Abweichungen gegenüber dem Projektantrag traten nicht auf. Das Projekt wurde wie angemeldet durchgeführt. Das Projektziel wurde erreicht und der Kundenauftrag erfüllt.

Abweichungen gegenüber der Projektplanung betrafen den Umfang und teilweise die Abfolge einzelner Vorgänge. Die Abweichungen bzgl. der Dauer wurden in Tabelle 2 erfasst:

Nr	Vorgang	Dauer (SOLL)	Dauer (IST)	Abw.
1.0	Analyse und Planung (15 Std.)	14,0	14,0	0,0
1.1	Ist-Analyse: DOSB-Bestandserhebung.	2,0	2,0	0,0
1.2	Ist-Analyse: BestandsModul, Tabellen und Exporte	2,0	2,0	0,0
1.3	Soll-Analyse: Anforderungen / Kundenwünsche	2,0	2,0	0,0
1.4	Analyse betroffener Tabellen / benötigte SQL-Befehle	2,0	2,0	0,0
1.5	Analyse Datenzugriffsmethoden des Frameworks	3,0	3,0	0,0
1.6	Konzeption und Planung grober Programmablauf	3,0	3,0	
2.0	Durchführung und Implementierung (32 Std.)	28,0	25,0	0,0
				-3,0
2.1	Erstellung der Datei-Vorlage in Excel	2,0	3,0	1,0

2.2	Programmierung des Datei-Imports (ImportStarter)	6,0	6,0	0,0
2.3	Programmierung der Datenverarbeitung in BOL	10,0	10,0	0,0
2.4	Programmierung der Aggregation nach Altersgruppen	6,0	-4,0	2,0
2.5	Integrationstest und Deployment	4,0	4,0	
3.0	Qualitätskontrolle und Abnahme (14 Std.)	14,0	12,0	0,0
				-2,0
3.1	Qualitätssicherung und begleitende Tests	6,0	6,0	0,0
3.2	Ergebniskontrolle	2,0	2,0	0,0
3.3	Soll-Ist-Vergleich	1,0	1,0	0,0
3.4	Test mit Pilotkunden	2,0	2,0	0,0
3.5	Abnahme durch Auftraggeber	1,0	1,0	0,0
3.6	Nacharbeiten	2,0	0,0	
4.0	Projekt- und Kundendokumentation (14 Std.)	14,0	19,0	-2,0
				5,0
4.1	Kundendokumentation	2,0	2,0	0,0
4.2	Anwenderdokumentation, Kurzeinführung für Kollegen	2,0	2,0	0,0
4.3	Projektdokumentation	10,0	15,0	
	Projektaufwand gesamt	70,0	70,0	5,0
				0,0

Tab. 2: Abweichungen Projektdauer SOLL - IST.

Für die Konzeptphase wurden einzelne Analyse-Schritte detaillierter als notwendig als Vorgänge geplant. Teils fielen Analyse-Schritte und Implementierung zeitlich auch zusammen. Nacharbeiten (nach Kundenvalidierung des Importformats) waren hingegen nicht schon in der Konzeptionsphase eingeplant, wurden aber durch den Zeitpuffer des gleichartigen Vorgangs der Durchführungsphase kompensiert.

Geringer als erwartet, fiel der Zeitaufwand für die Programmierung der Aggregation nach Altersgruppen aus, weil hierüber anfangs wenig Klarheit bestand und entsprechend mehr Zeit dafür eingeplant wurde. Letztlich konnte eine bereits im System existierende Methode dafür genutzt und Zeit eingespart werden. Die gewonnene Zeit konnte zusätzlich für die Projektdokumentation verwendet werden, die umfangreicher geriet, als ursprünglich erwartet.

4.3 Qualitätskontrolle

Die Projektarbeit erfüllt ihren Zweck und den geforderten Funktionsumfang an das Importprogramm. Die Fertigstellung erfolgte zudem in der vorgesehen Zeit und somit im kalkulierten (Zeit)Budget. Die Frühzeitige Rückkopplungsschleife mit dem Pilotkunden war eine sinnvolle Entscheidung, um das Programm kundenorientiert und praxistauglich zu entwickeln. Mit der erstellten Anwenderdokumentation ist die Programmnutzung auch ausreichend dokumentiert.

Weitere wesentliche Faktoren zur Qualitätssicherung in diesem Projekt waren die Beachtung gängiger Entwicklungsprinzipien und Best Practices: Der Einsatz von Transaktionen sollte

dabei im Datenbankumfeld ebenso selbstverständlich Anwendung finden, wie auch das Abfangen von potentiellen Ausnahmefehlern mit Try-Catch-Strukturen im Programmcode.

Auch die durchgängige Nutzung der Framework-Unterstützung trug zur Entwicklung soliden Programmcodes bei, was speziell für die Datenzugriffsoperationen hohe Qualität sicherstellte. Besonders zu schätzen gelernt hatte der Projektbearbeiter dabei auch die Nutzung typisierter *DataSets*, wodurch Fehler durch Datentyp-Inkompatibilitäten oder aufgrund von Vertippern gar nicht erst aufkommen konnten. Die damit einhergehende IntelliSense-Unterstützung sorgte zusätzlich für Entwicklungskomfort, Qualität und Effizienz.

4.4 Fazit

Das persönliche Fazit zur betrieblichen Projektarbeit fällt uneingeschränkt gut aus: Es war lehrreich - hinsichtlich frühzeitiger Kundeneinbindung und Nutzung von *DataView*-Objekten – und gut zu bewältigen, mit interessanten Herausforderungen bei der bedingten Ablaufsteuerung für Lese- und Schreibzugriffe sowie bei der Durchlaufzeit-Optimierung.

Die Begleitung durch den Projektbetreuer war hervorragend und jederzeit auch die Möglichkeit gegeben, Unterstützung durch Entwickler-Kollegen oder den Auftraggeber zu erhalten.

4.5 Ausblick / Zukunft

Daten sind das Kernstück vieler Software-Systeme. Die elektronische Datenverarbeitung ist – trotz der vermeintlich angestaubten Bezeichnung – ein fortwährend aktuelles Thema auch in den Geschäftsprozessen des 21. Jahrhunderts. Die *IntelliVerband*-Software bietet Sportverbänden hier optimale Unterstützung zur Verwaltung der Mitglieder und Geschäftsprozesse. Die Integration von Daten aus anderen Systemen wird dabei immer selbstverständlicher von Kunden erwartet und eingefordert. Im diesen Sinne ist auch zukünftig mit DatenimportProjekten zu rechnen, um weitere Schnittstellen für andere Module zu realisieren.

5. QUELLENVERZEICHNIS

- [1] DOSB: Bundesweit einheitliche Regelungen zur Zuordnung zu Fachverbänden. URL: <https://www.dosb.de/de/service/download-center/dosb-organisation/bestandserhebung/>.
Dokument: Bundesweit_einheitliche_Regelungen_zur_Zuordnung_zu_Fachverbänden.pdf
- [2] DOSB: Vereinsverwaltungssoftware. Übersicht von Softwareprodukten zur Vereinsverwaltung. URL: <https://www.dosb.de/de/service/download-center/schnittstelle-bestandsdaten/> Dokument: Vereinsverwaltungssoftware.pdf
- [3] WIKIPEDIA – Die freie Enzyklopädie. URL: <https://de.wikipedia.org/>, Diverse Artikel zur informationstechnischen Themen.
- [4] MSDN – Microsoft Developer Network. URL: <https://msdn.microsoft.com/de-de/>
Informationen zu Syntax und Beispielen in VB.NET und MS SQL-Server 2012 (T-SQL).

[5] DOBERENZ, W.;GEWINNUS, T.: Datenbank-Programmierung mit Visual Basic 2012.
Grundlagen, Rezepte, Anwendungsbeispiele. Microsoft Press. O'Reilly Verlag, Köln, 2013.

Außer den angegebenen Quellen wurden keine weiteren Hilfsmittel benutzt.

Die verwendeten Tabellen und Abbildungen wurden alle selbst erstellt mit unterschiedlichen
Programmen aus dem Microsoft Office Paket: Word, Excel, PowerPoint, Visio oder Project.

Code-Ausschnitte und Datenbankdiagramm stammen aus der verwendeten
Entwicklungsumgebung: Visual Studio Professional 2012 und SQL Server 2012 Management
Studio.

Nr	Vorgang	Dauer	kw 40							kw 41							kw 42							kw 43							kw 44							kw 45						
			Mo 14.	Mo 28.	Di 29.	Mi 30.	Do 01.	Fr 02.	Sa 03.	So 04.	Mo 05.	Mo 06.	Di 07.	Mi 08.	Do 09.	Fr 10.	Sa 11.	Mo 12.	Mo 13.	Di 14.	Mi 15.	Do 16.	Fr 17.	Sa 18.	Mo 19.	Mo 20.	Di 21.	Mi 22.	Do 23.	Fr 24.	Sa 25.	Mo 26.	Mo 27.	Di 28.	Mi 29.	Do 30.	Fr 31.	Sa 01.	Mo 02.	Mo 03.	Di 04.	Mi 05.	Do 06.	Fr 07.
0.1	M 0: Projektantrag genehmigt	0,0																																										
1.0	Analyse und Planung (15 Std.)	14,0																																										
1.1	Ist-Analyse: DOSB-Bestandshebung.	2,0																																										
1.2	Ist-Analyse: BestandsModul, Tabellen und Exporte	2,0																																										
1.3	Soll-Analyse: Anforderungen / Kundenwünsche	2,0																																										
1.4	Analyse betroffener Tabellen / benötigte SQL-Befehle	2,0																																										
1.5	Analyse Datenzugriffsmethoden des Frameworks	3,0																																										
1.6	Konzeption und Planung grober Programmablauf	3,0																																										
1.7	M 1: Analyse und Planung abgeschlossen	0,0																																										
2.0	Durchführung und Implementierung (32 Std.)	28,0																																										
2.1	Erstellung der Datei-Vorlage in Excel	2,0																																										
2.2	Programmierung des Datei-Imports (ImportStarter)	6,0																																										
2.3	Programmierung der Datenverarbeitung in BOL	10,0																																										
2.4	Programmierung der Aggregation nach Altersgruppen	6,0																																										
2.5	Integrationstest und Deployment	4,0																																										
2.6	M 2: Implementierung abgeschlossen	0,0																																										
3.0	Qualitätskontrolle und Abnahme (14 Std.)	14,0																																										
3.1	Qualitätssicherung und begleitende Tests	6,0																																										
3.2	Ergebniskontrolle	2,0																																										
3.3	Soll-Ist-Vergleich	1,0																																										
3.4	Test mit Pilotkunden	2,0																																										
3.5	Abnahme durch Auftraggeber	1,0																																										
3.6	Nacharbeiten	2,0																																										
3.7	M 3: Ergebnisse kontrolliert und abgenommen	0,0																																										
4.0	Projekt- und Kundendokumentation (14 Std.)	14,0																																										
4.1	Kundendokumentation	2,0																																										
4.2	Anwenderdokumentation, Kurzeinführung für Kollegen	2,0																																										
4.3	Projektdokumentation	10,0																																										
4.4	M 4: Dokumentation abgeschlossen	0,0																																										
	Projektaufwand gesamt	70,0																																										

Legende:

- Vorgang für diesen Tag geplant
- Tage wg. Blockunterricht nicht verfügbar
- Wochenende / Feiertage

Abb. 10 Projektablauf (Gantt -Diagramm)

B. Datenmodell Bst-Modul

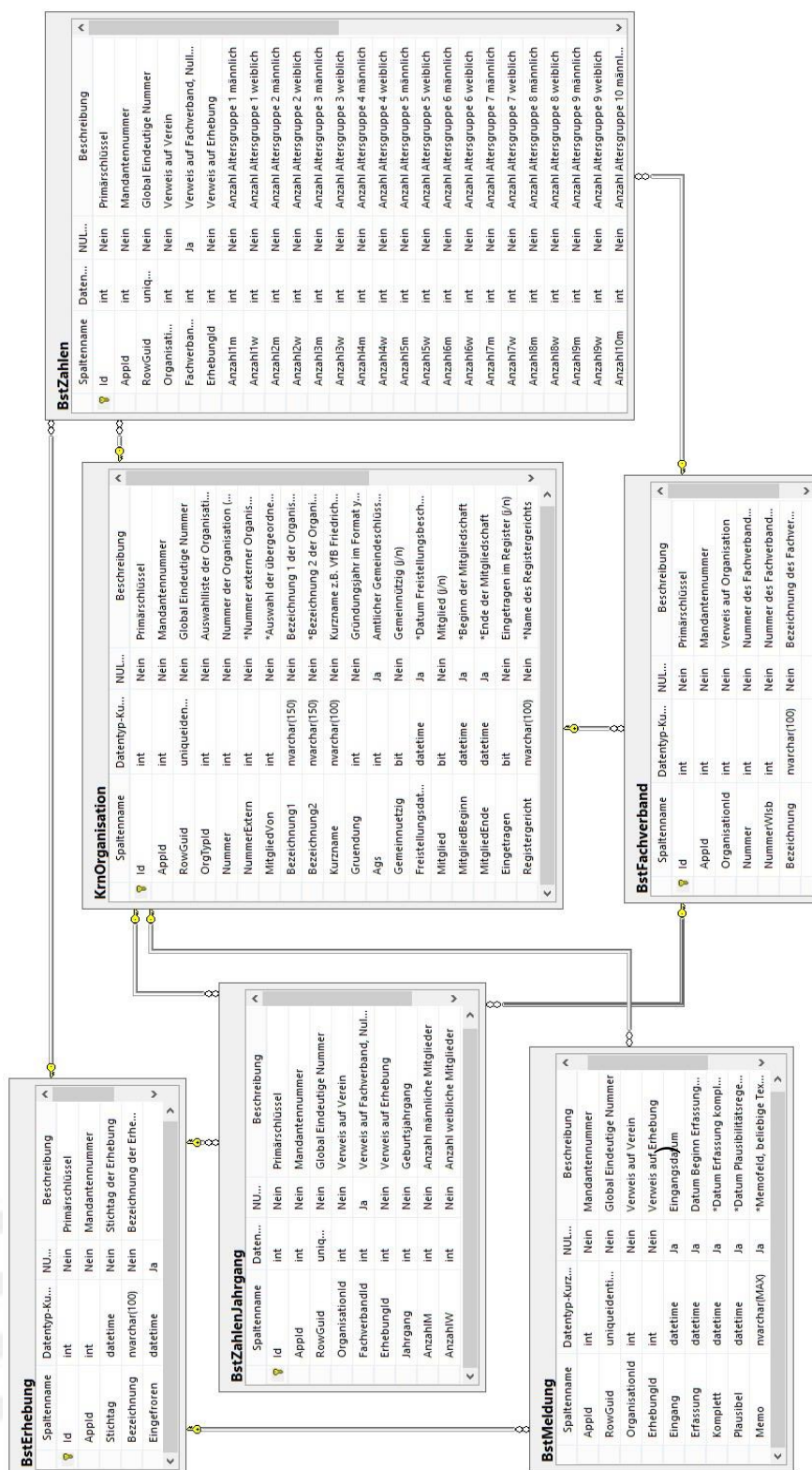


Abb. 11 Datenmodell Bestandsdaten (Ausschnitt)

C. Anwenderdokumentation



Inhalt

1. VORBEMERKUNG	3
2. IMPORTDATEI VORBEREITEN	3
3. GRUNDLAGENDATEN (LOOKUP-TABELLEN)	3
3.1 ORGANISATIONEN.....	3
3.2 FACHVERBÄNDE	3
3.3 ERHEBUNG	4
4 IMPORT-PROGRAMM.....	4
4.1 .CONFIG EINSTELLUNGEN	4
4.2 PARAMETER UND ARGUMENTE	4
4.3 ABLAUF.....	5
4.2.1 ImportJob initialisieren	5
4.2.2 ImportJob Starten	6
5. TROUBLE SHOOTING	6
5.1 VERARBEITUNGSFEHLER – APPLOG.....	6
5.2 FEHLER BEIM PROGRAMMAUFRUF	6
6. ANHANG: IMPORTFORMAT ERLÄUTERUNGEN.....	7
6.1 BESTANDSMELDUNG – JAHRGANG IN SPALTEN	7
6.2 BESTANDSMELDUNG – JAHRGANG IN ZEILEN	8

1. VORBEMERKUNG

Für den Bestandsdaten-Import existieren zwei Formatvorlagen in Excel auf dem SharePoint unter SportBizz > TeamDokumente > Support > Datenmigration:

- 1) *Importformat Bestandsmeldung Spalten.xls*
- 2) *Importformat Bestandsmeldung Zeilen.xls*

Das Importformat 1) enthält pro Zeile die Meldung für einen Verein und einen Fachverband; die Anzahl der Mitglieder pro Jahrgang (m/w) stehen in den folgenden 220 *Spalten*. Dieser Tabellenaufbau ist ähnlich dem *IntelliVerband-Export "Auswertung 11"*.

Das Importformat 2) enthält pro Zeile die Mitgliederanzahlen (m/w) für einen Jahrgang; das bedeutet pro Verein und Fachverband stehen die Zahlen einer Meldung in bis zu 110 *Zeilen*. Dieser Tabellenaufbau ist mitunter für Fremdsysteme leichter zu generieren, erzeugt aber auch entsprechend große Dateien mit vielen Zeilen.

Hinweis: Aktuell ist nur die Importroutine für das Tabellenformat unter 2) ausprogrammiert.

Inhaltliche Bedeutung der Felder sowie Datentypen, Beispielwerte und Infos zu Pflichtfeldern sind im ebenfalls auf dem SharePoint verfügbaren *Importformat Erläuterungen.docx* beschrieben (Auszug im Anhang).

2. IMPORTDATEI VORBEREITEN

Für die Übernahme von Mitgliederbestandszahlen aus einem Fremdsystem obliegt es dem Kunden, eine Excel-Datei mit den Daten im vorgegebenen Format zu befüllen. Pflichtfelder, Default-Werte und Datentyp-Vorgaben gemäß den *Importformat Erläuterungen* sind zu berücksichtigen und vor dem Import durch Augenscheinnahe zu prüfen.

Wichtig: Die Importdatei ist im alten .xls Format zu speichern. Die daraus resultierende Beschränkung der Tabellen auf 65.536 Zeilen ist zu beachten. Bei größeren Landesverbänden kann hier bereits bei einer Erhebung der Grenzwert überschritten und somit eine Aufteilung auf zwei Dateien nötig sein.

3. GRUNDLAGENDATEN (LOOKUP-TABELLEN)

3.1 Organisationen

Die Importroutine erwartet, dass die Vereine, für die Mitgliederbestandsdaten eingelesen werden sollen, in der Datenbank bereits existieren (Tabelle: *KrmOrganisation*).

Sollte erst noch ein Stammdaten-Import der Organisationen nötig sein, so steht dafür das → *Importformat Organisation* mit entsprechender Importroutine zur Verfügung.

3.2 Fachverbände

Des Weiteren muss eine Liste mit den in der Importdatei vorkommenden Fachverbände und deren Nummern in der Datenbank existieren (Tabelle: *BstFachverband*).

Diese ist ggfs. als einfache Liste (Nummer, Fachverband, NummerDOSB) vom Kunden zu erfragen und direkt per SQL in die Datenbank zu schreiben. Dabei ist eine Verknüpfung des abstrakten Fachverbands zur konkreten Fachverbandorganisation im Kemmodul über die OrganisationId herzustellen.

3.3 Erhebung

Als Drittes muss auch die Erhebung bereits existieren, für die die Bestandsmeldungen eingelesen werden sollen (Tabelle: *BstErhebung*).

Eine Erhebung wird meist noch nicht vorhanden sein und ist dann per SQL in der Datenbank direkt oder über die Benutzeroberfläche des Portals, im Bereich *Bestandserhebung* > *Grundlagen* > *Erhebungen* > *Neu* anzulegen.

4 IMPORT-PROGRAMM

Das Programm zum Start des Importvorgangs ist eine einfache Konsolenanwendung.

Zur Durchführung des Bestandsdatenimports in eine lokale Datenbank, kann die Solution *IWeb.SportBiz.Krn.Import* in *Visual Studio* aufgerufen und das Programm direkt aus der Entwicklungsumgebung heraus gestartet werden.

Findet der Importvorgang in ein Produktivsystem des Kunden statt, so muss das Projekt zuvor kompiliert werden (*Visual Studio* > *Projektmappe neu erstellen*) und die resultierende *IWeb.SportBiz.Krn.Import.exe* auf dem Zielsystem ausgeführt werden.

4.1 .config Einstellungen

Bevor der Import gestartet werden kann ist der Connection-String für die Datenbank in der Datei *app.config* auf die aktuelle Umgebung anzupassen:

```
<add key="sqlconnection" value="Data Source=.;Initial  
Catalog=SportBizzLsbh;Integrated Security=True" />
```

Arbeitet man auf dem Zielsystem mit der kompilierten *.exe*, so ist der Connection-String entsprechend in der Datei *IWeb.SportBiz.Krn.Import.exe.config* anzupassen.

4.2 Parameter und Argumente

Das Programm erwartet einen der nachfolgenden Parameter (i, b, o, w) von einem / und : eingeschlossen und unmittelbar gefolgt vom entsprechenden Befehlsargument.

```
Select Case cmd  
Case "/i:" ' arg = FileName  
    InitImportJob(arg)  
Case "/o:" ' arg = JobId  
    StartJobKrnOrganisation(Integer.Parse(arg))  
Case "/b:" ' arg = JobId  
    StartJobBstMeldung(Integer.Parse(arg))  
Case "/w:" ' arg = ImageId  
    WriteImage(Integer.Parse(arg))
```

4.3 Ablauf

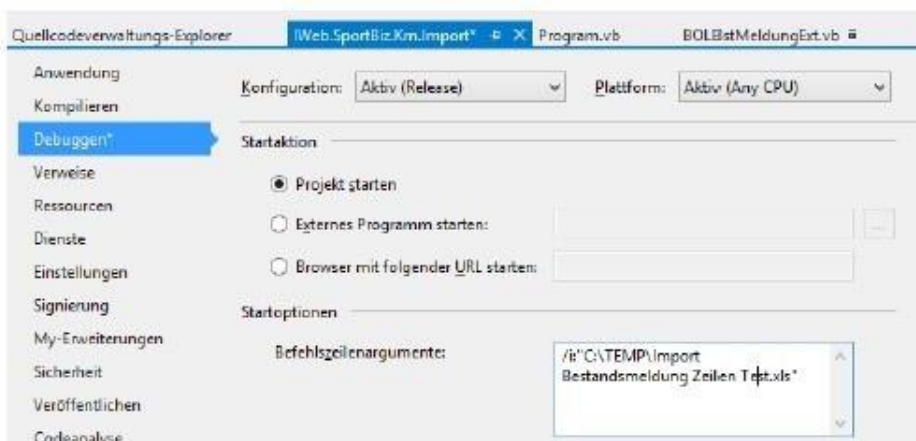
Gestartet wird das Programm über die Eingabeaufforderung oder direkt aus Visual Studio mit Einstellung der Parameter in den Eigenschaften (Debuggen > Startoptionen) des Projekts.

4.2.1 ImportJob initialisieren

Zur Initialisierung des Imports wird Parameter `/i:` verwendet und der Dateiname inklusive Pfad als Befehlsargument nach dem Doppelpunkt angegeben. Enthält der Pfad oder der Dateiname Leerzeichen so ist der ganze Pfad in Anführungszeichen zu setzen.

```
/i:"C:\TEMP\Import Bestandsmeldung Zeilen.xls"
```

In Visual Studio lassen sich der Parameter mit Befehlsargument in den Eigenschaften des Projekts, im Bereich Debuggen unter Startoptionen einstellen:



Alternativ ist bei Aufruf über die Kommandozeile (*Ausführen der .exe als Administrator!) der komplette Programmname `IWeb.SportBiz.Krn.Import.exe` mit samt Parameter und Befehlsargument einzugeben:



Die Inhalte der Excel-Datei werden als Binärdaten (Image) in der Datenbank gespeichert.

Gleichzeitig wird in der Datenbank, in der Tabelle `AppJob` ein entsprechender Eintrag mit Verweis auf das Image eingefügt. Die Id dieses Jobs wird für den folgenden Aufruf benötigt.

4.2.2 ImportJob Starten

Die zuvor ausgeführte Prozedur schreibt einen Eintrag in die Tabelle *AppJob* der Datenbank. Die dort eingetragene Id des Jobs wird im nächsten Schritt als Übergabeparameter genutzt (im Beispiel für JobId = 3).

`/b:3`

Parameter `/b`: ruft nun die Prozedur `StartJobBstMeldung` auf, womit die Verarbeitung der zuvor als Image (Binärdaten) abgespeicherten Excel-Tabelle beginnt.

Nach Abschluss des ImportJobs wird in der Tabelle *AppLog* der Status des Jobs aktualisiert. Detailliertere Meldungen über die erfolgreiche oder fehlerhafte Verarbeitung werden zudem in der Tabelle *AppLog* unter derselben JobId protokolliert.

Hinweis: Neben dem eigentlichen Importvorgang, der die Mitgliederbestandsdaten in die Tabelle *BstZahlenJahrgang* schreibt, findet in der selben Transaktion auch die Aggregation nach Altersgruppen (Tabelle *BstZahlen*) statt. Sollte auch nur letzteres fehlschlagen, wird der Importvorgang insgesamt zurückgerollt.

5. TROUBLE SHOOTING

5.1 Verarbeitungsfehler – AppLog

Das *AppLog* gibt Aufschluss über aufgetretene Verarbeitungsfehler. Meist sind diese in fehlenden Fremdschlüsselreferenzen (vgl. 3. Lookup-Tabellen) oder durch Missachtung der definierten Pflichtfelder bzw. die Verwendung falscher Datentypen (vgl. 6. Importformat Erläuterungen) begründet. Ein Fehler bei der Summenbildung für die Altersgruppen kann hierbei nur bei fehlender Altersgruppendefinition in der Tabelle *AppValues* auftreten.

5.2 Fehler beim Programmaufruf

Tritt bereits beim Aufruf der exe-Datei auf dem Remote-Server eine Fehlermeldung auf, so gibt es hierfür zwei typische Ursachen:

- 1) Die Hilfsbibliothek `IWeb.Tools.UserHelperWeb.dll` ist auf dem Zielsystem nicht richtig installiert bzw. der Eintrag in der Datenbank verweist auf einen falschen Pfad.
 - ➔ Dies ist in der Tabelle *AppValues* (Filter im Feld *ValueKey* auf `%UserHelper%`) zu überprüfen und ggfs. hier neu zu setzen. Alternativ: Die `.dll` aus dem `ExtLib`-Verzeichnis kopieren und im entsprechenden Verzeichnispfad abspeichern.
- 2) Der "OLEDB-Provider ist auf dem System nicht registriert". Diese Fehlermeldung tritt auf 32bit-Systemen auf, wenn das Importprogramm auf einem 64bit-System ohne spezielle Einstellung für die Ziel-CPU kompiliert wurde.
 - ➔ Dies ist im *Visual Studio* in den Eigenschaften der Solution unter Kompilieren (Ziel-CPU: x86) zu überprüfen bzw. einzustellen und die Projektmappe dann nochmals neu zu erstellen und auf dem Zielsystem erneut auszuführen.

6. ANHANG: IMPORTFORMAT ERLÄUTERUNGEN

6.1 Bestandsmeldung – Jahrgang in Spalten

Bezeichnung des Registerblatts: *BestandsmeldungSpalten*

Erläuterung zu den Spalten (Pflichtfelder farblich markiert):

Spaltenname	Datenbank	Typ	Leer	Beispiel	Erläuterung
Nummer	OrganisationId	Zahl	Nein	10121	Nummer des Vereins, der die Zahlen meldet. Umwandlung in Fremdschlüssel (KrnOrganisation).
ErhebungStichtag	ErhebungId	Datum	Nein	01.01.2015	Stichtag der Erhebung, üblicherweise der 01.01. eines Jahres. Umwandlung in Fremdschlüssel (BstErhebung).
Fachverband	FachverbandId	Zahl	Nein	86	Nummer des Fachverbands laut DOSB-Liste (B-Meldung), 0 für A-Meldung. Umwandlung in Fremdschlüssel (BstFachverband).
DatumEingang	Eingang	Datum	Nein	15.03.2015	Eingangsdatum (Sendedatum aus Online-System).
DatumErfassung	Erfassung	Datum	Nein	15.03.2015	Datum bei Erfassung / Übernahme ins verarbeitende System.
DatumKomplett	Komplett	Datum	Nein	15.03.2015	Datum der erfolgreichen Prüfung auf Vollständigkeit.
DatumPlausibel	Plausibel	Datum	Nein	15.03.2015	Datum der erfolgreichen Prüfung auf Plausibilität.
MemoMeldung	Memo	Text	Ja	Notizen	<Null> wenn leer.
AnzahlAlter10M (0M ... 110M)	AnzahlIM + Jahrgang	Zahl	Ja	8	Anzahl männliche Mitglieder je Alter (10M = Alter 10 Jahre*, männlich), 0 wenn leer. *Umrechnung Alter in entsprechenden Jahrgang = Jahr der Erhebung - 1 - Alter)
AnzahlAlter10W (0M ... 110W)	AnzahlIW + Jahrgang	Zahl	Ja	11	Anzahl weibliche Mitglieder je Alter (10W = Alter 10 Jahre*, weiblich), 0 wenn leer. *Umrechnung Alter in entsprechenden Jahrgang = Jahr der Erhebung - 1 - Alter)
Insert in Tabellen:	BstMeldung, BstZahlenJahrgang, (BstErhebung)				
Lesen in Tabellen:	KrnOrganisation, BstErhebung, BstFachverband, (BstMeldung)				

6.2 Bestandsmeldung – Jahrgang in Zeilen

Bezeichnung des Registerblatts: *BestandsmeldungZeilen*

Erläuterung zu den Spalten (Pflichtfelder farblich markiert):

Spaltenname	Datenbank	Typ	Leer	Beispiel	Erläuterung
Nummer	OrganisationId	Zahl	Nein	10121	Nummer des Vereins, der die Zahlen meldet. Umwandlung in Fremdschlüssel (KrnOrganisation).
ErhebungStichtag	ErhebungId	Datum	Nein	01.01.2015	Stichtag der Erhebung, üblicherweise der 01.01. eines Jahres. Umwandlung in Fremdschlüssel (BstErhebung). Erzeugung einer neuern Erhebung wenn leer.
Fachverband	FachverbandId	Zahl	Nein	86	Nummer des Fachverbands laut DOSB-Liste (B-Meldung), 0 für A-Meldung. Umwandlung in Fremdschlüssel (BstFachverband).
DatumEingang	Eingang	Datum	Nein	15.03.2015	Eingangsdatum (Sendedatum aus Online-System).
DatumErfassung	Erfassung	Datum	Nein	15.03.2015	Datum bei Erfassung / Übernahme ins verarbeitende System.
DatumKomplett	Komplett	Datum	Nein	15.03.2015	Datum der erfolgreichen Prüfung auf Vollständigkeit.
DatumPlausibel	Plausibel	Datum	Nein	15.03.2015	Datum der erfolgreichen Prüfung auf Plausibilität.
MemoMeldung	Memo	Text	Ja	Notizen	<Null> wenn leer.
Jahrgang	Jahrgang	Zahl	Nein	1984	Jahrgang als Zahl
AnzahlM	AnzahlM	Zahl	Ja	8	Anzahl männliche Mitglieder in diesem Jahrgang, 0 wenn leer.
AnzahlW	AnzahlW	Zahl	Ja	11	Anzahl weibliche Mitglieder in diesem Jahrgang, 0 wenn leer.

Insert in Tabellen: BstMeldung, BstZahlenJahrgang, (BstErhebung)
Lesen in Tabellen: KrnOrganisation, BstErhebung, BstFachverband, (BstMeldung)

D. Persönliche Erklärung



Persönliche Erklärung zur betrieblichen Projektarbeit

Hiermit versichere ich, Herr M
(Vor- und Zuname des Auszubildenden)

die betriebliche Projektarbeit Konzeption und Entwicklung einer Schnittstelle zum Datenimport von über Drittanbieter-Systeme erfassten Mitgliederbestandsmeldungen in das IntelliVerband-Portal
(genaue Bezeichnung der Projektarbeit)

sowie die eingereichte Dokumentation unter der Betreuung von

Herr x / xyz GmbH
(Vor- und Zuname des Ausbilders/Name des Ausbildungsbetriebes)

selbstständig und ohne fremde Hilfe konzipiert, verfasst und durchgeführt zu haben. Teile der Dokumentation, die ich nicht selbstständig erstellt habe, sind von mir entsprechend gekennzeichnet worden.

Die von der Verordnung vorgesehene Richtzeit wurde eingehalten.

Ich bin darüber aufgeklärt worden, dass meine betriebliche Projektarbeit bei Täuschungshandlungen bzw. Ordnungsverstößen mit „null“ Punkten bewertet wird und als nicht bestanden gilt.

Ich bin weiter darüber aufgeklärt worden, dass dies auch dann gilt, wenn festgestellt wird, dass meine Projektdokumentation im Ganzen oder zu Teilen mit der eines anderen Prüfungsteilnehmers übereinstimmt.

Ich nehme zur Kenntnis, dass ggf. stichprobenartige Kontrollen durchgeführt werden können.

Musterstadt, 30.2.1916 unleserlich
(Ort und Datum) (Unterschrift des Prüfungsteilnehmers)

Xrabel
(Unterschrift des Ausbilders/Stempel)

Diese Erklärung ist der Dokumentation beizufügen.

Stand: November 2014/GB 2/gr

1 von 1